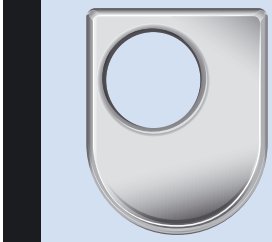

M363 Unit 14
UNDERGRADUATE COMPUTING

Software engineering with objects

Human factors and professional issues

Unit 14

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 2192 8

1.1

CONTENTS

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

This unit explores the human factors of software development at the level where people design and build the software. At every level the people involved in the software development process have a major influence on its outcome. In the first half of this unit, you will learn how the human environment affects software professionals, as well as what it means to be part of a profession. The second half of this unit examines ethical decision making and how to approach situations involving ethical issues, the legal responsibilities and rights of software professionals, and the role of professional bodies.

2 Introducing human and professional issues

Software is now used in so many devices that few are untouched by it. Software is essential to the working of many organisations through their administrative and financial systems, and through other systems that give them competitive advantage. Software is contained in microprocessors embedded in many engineering artefacts, from motor vehicles to videos.

The development of software involves software engineers: people working for an organisation, in order to produce software which will be acquired by other organisations, for use by other people either directly or indirectly. The terms used to describe such people vary from organisation to organisation and include software designer, analyst, software engineer and sometimes just programmer. In this unit we shall use the term **software engineer** as the generic term to describe a person working in software development, and shall occasionally abbreviate this to *engineer*.

Contrary to an impression often given, most software does work adequately and provides a valuable service to its users. The engineers who develop it usually work well and successfully. But occasionally things do go wrong. From the earliest days of data processing there has been evidence of substandard software performance: for example, cheques have been issued for £0.00 and multiple copies of invoices have been sent out. These problems continue today, but they are not all so trivial. Perhaps multiple copies of computer-generated junk mail are an acceptable hazard of modern society, but unreliable computerised systems to control air traffic or nuclear power stations are clearly not acceptable. The economic arguments for extending the use of computers into these and other safety-critical areas are very powerful, and the fact that the potential is not always matched by the results has not deterred those advocating the use of software systems – they continue to believe that improved software development processes will cut the risks to acceptable levels.

There are many examples of computer-related accidents. One of the classic case studies is the Therac-25 software-controlled radiation therapy machine, which was responsible for the death of several patients in the late 1980s. We shall use this example to lead into consideration of the importance of human factors and professionalism in the development of software.

See Leveson and Turner (1993, pp. 18–41).

The results of a thorough investigation into the Therac-25 machine were reported by Nancy Leveson and Clark Turner. Eleven of these machines were installed, five in the US and six in Canada. Six accidents occurred between 1985 and 1987 involving massive radiation overdoses, and the machines were recalled in 1987. Software errors were detected, as well as other problems related to system use and hardware.

Each Therac-25 machine had a turntable that rotated equipment into the X-ray beam to produce different modes of operation. A computer was responsible for positioning the turntable. The operator positioned the patient, set the treatment parameters and placed the equipment on the turntable by hand. The operator then entered the information at a console, and the system compared the values set manually with the values entered at the console.

Three flaws in the system were identified.

1 The operator could edit a command line to change the state of the machine such that the execution of the radiation commands took place before the change in the state of the machine was complete. So, after the operator entered the treatment data – mode, energy level, position, and so on – he or she could then alter the data,

by moving the cursor to the field to be changed, without having to start the data entry procedure from the beginning. If this was done too quickly, the new values were shown on the screen, but the internal treatment parameters remained unchanged.

2 Safety checks were bypassed when a program counter reached zero. This counter, represented by a variable of one byte, was incremented by one every time a test was performed. If its value was zero, no safety tests would be performed. Since the variable was only one byte, it could contain a maximum value of 255; so, every 256th pass, the variable overflowed, had a zero value and no safety tests were performed.

3 Hardware safety interlocks had been removed from the machine because they were supposed to be done in software. The machine was made to rely entirely on software for safety.

In their report of the investigation, Leveson and Turner reflected on the lessons that should be learnt:

> Most previous accounts of the Therac-25 accidents blamed them on a software error and stopped there. This is not very useful, and, in fact, can be misleading and dangerous: If we are to prevent such accidents in the future, we must dig deeper. Most accidents involving complex technology are caused by a combination of organizational, managerial, technical, and, sometimes, sociological or political factors. Preventing accidents requires paying attention to *all* the root causes, not just the precipitating event in a particular circumstance.
>
> Accidents are unlikely to occur in exactly the same way again. If we patch only the symptoms and ignore the deeper underlying causes or we fix only the specific cause of one accident, we are unlikely to prevent or mitigate future accidents. The series of accidents involving the Therac-25 is a good example of exactly this problem: Fixing each individual software flaw as it was found did not solve the device's safety problems. Virtually all complex software will behave in an unexpected or undesired fashion under some conditions – there will always be another bug. Instead, accidents must be understood with respect to the complex factors involved. In addition, changes need to be made to eliminate or reduce the underlying causes and contributing factors that increase the likelihood of accidents or of loss resulting from them.
>
> Although these accidents occurred in software controlling medical devices, the lessons apply to all types of systems where computers control dangerous devices. In our experience, the same types of mistakes are being made in nonmedical systems. We must learn from our mistakes so we do not repeat them.
>
> (Leveson and Turner, 1993)

How can disasters like this be avoided? Mistakes are made by humans, the humans who develop the software and systems. To try to avoid mistakes, we need to understand more fully the human needs of software engineers. These *human factors* are introduced in the next subsection and elaborated in Sections 3 and 4.

We also expect the software engineers to behave responsibly, and for that we turn to the concept of *professionalism*. This concept is discussed in the subsection on 'Being professional', and is an important component of ethics, discussed later in this unit.

2.1 Human factors and software development

Human factors is a discipline that tries to enhance the relationship between people and technology, where the word 'enhance' means making the relationship safer or more productive. In software development, we tend to think of the people–technology

relationship in terms of the end-user of a software system: for example, a pilot in a cockpit, surrounded by displays and flashing lights. The human factors involved in this view of the people–technology relationship are called **product-related human factors**, and have long been recognised as an important consideration in the design of interactive software.

However, the software development process involves another people–technology relationship: namely software engineers, at their desks, using computers to visualise, specify, design and construct software products. Therefore, human factors have a bearing on both the software development *product* (because it is a technological artefact that supports human activities) and the software development *process* (because it involves people and technology). There are not only product-related human factors but also **process-related human factors**. We shall focus in this unit on process-related human factors, in particular on how software engineers contribute to the efficiency of the process and the quality of the product.

2.2 Process-related human factors

Human beings are complex entities: they have many characteristics and engage in many different behaviours. Not all of these characteristics and behaviours will have any bearing on software engineering. For example, it is unlikely that the length of a software engineer's forearm will affect the ability to develop a requirements specification, although it might well be of interest if the engineer retrained as a fighter pilot! The focus here is on identifying and grouping the human factors that are most relevant to software engineering.

Consider the software development office shown in Figure 1. Focus on the young software engineer in the foreground, working with a specification to implement a code module. How many things might affect how quickly and how accurately the engineer performs this task?

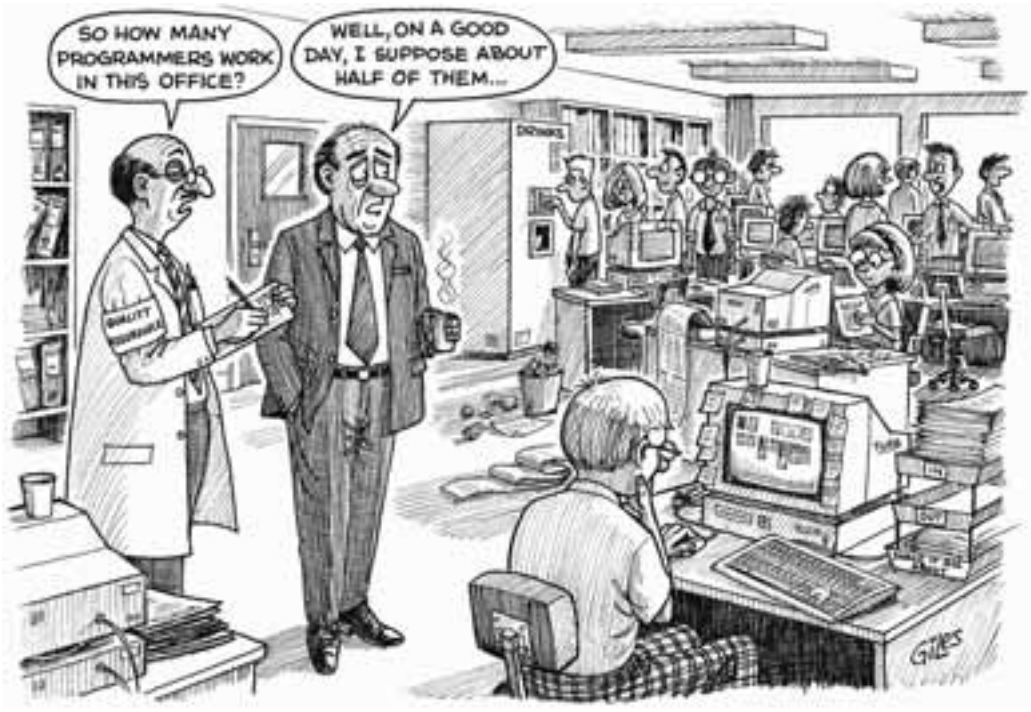

Figure 1 A software development office

The engineer's performance might be affected by, for example, the following factors:

1 the engineer's experience of programming;
2 the engineer's understanding of the specification;
3 stress in the engineer's personal life;
4 the number of other tasks the engineer has to do;
5 the similarity between this code module and others the engineer has implemented;
6 the usability of the debugging tools;
7 the length of time the engineer can expect to work without interruption;
8 the layout and clutter of the engineer's desk;
9 noise in the office.

All of the process-related human factors listed here arise either from the characteristics and skills of the engineer or are external influences that have an impact on how effectively they can apply themself to the task. The factors can be divided into three categories:

- person-related;
- task-related;
- environment-related.

Person-related human factors

The following factors relate directly to the software engineer as a person:

1 the engineer's experience of programming;
2 the engineer's understanding of the specification;
3 stress in the engineer's personal life.

Task-related human factors

The following factors relate to the tasks the engineer is undertaking:

4 the number of other tasks the engineer has to do;
5 the similarity between this code module and others the engineer has implemented.

Environment-related human factors

The remaining four factors are neither person-related nor task-related:

6 the usability of the debugging tools;
7 the length of time the engineer can expect to work without interruption;
8 the layout and clutter of the engineer's desk;
9 noise in the office.

These factors are associated with the environment in which the engineer works, where 'environment' here means the *physical space* and the *tools* within that space.

The division of process-related human factors into different categories is useful because it allows us to concentrate on one aspect at a time. However, in reality this division is a simplification of the situation: human factors are rarely independent. More typically, there is an interaction between factors. For example, the engineer's experience of programming (a person-related factor) will help to determine the complexity of the code module being worked on (a task-related factor), as an inexperienced programmer can cope with complexity less easily and efficiently than an experienced one. However, we shall continue to use this division, and in Sections 3 and 4 we shall consider each category in some detail.

SAQ 1

How can a consideration of human factors improve the relationship between people and technology in the software development process?

ANSWER.....

First, by making the process of developing software more efficient and effective (that is, by analysing *process-related factors*). Second, by improving the interface between people and the final software product (that is, by analysing *product-related factors*).

SAQ 2

Imagine that the task of the software engineer in Figure 1 was to test, rather than to implement, the code module. How would the two task-related human factors listed above change?

ANSWER.....

The factors would change to reflect the specific task. For example, the number of other tasks the engineer has to do might remain as a factor, but the second factor would have to change to reflect the characteristics of the new testing task, as follows:

> the similarity between this test and others the engineer has run.

SAQ 3

How would you classify the factor 'the engineer's experience of the application domain'?

ANSWER.....

It is a person-related human factor.

2.3 Being professional

The word 'professional' is used in many contexts, ranging from sport to commerce and industry. It almost certainly has different shades of meaning for different people, and can carry emotive connotations. Consider, for example, how you might feel if your work was referred to as 'amateurish' instead of 'professional'.

The aim of this section is to explore what it means to be 'professional', in the context of work in general and software engineering in particular.

What does it mean to be 'professional'?

It is commonly accepted that if someone is referred to as 'professional,' it is a compliment. The individual has shown certain qualities in the way they approach and carry out their work that deserve praise. This is consistent with the definition of 'profession' from the *Longman Dictionary of Contemporary English* (1987):

> **profession**: a form of employment, esp. one that is possible only for an educated person and after training (such as law, medicine or teaching) and that is respected in society as honourable.

We might consider why a term has been coined to distinguish between these classes of occupation and others, as exemplified by the following definition from the same dictionary:

> **trade**: a job, especially needing a special skill with the hands: Being a printer is a trade; being a lawyer is a profession.

A key feature of the professions, be they traditional like the medical profession or new like software engineering, is that it is difficult for lay people to assess the ability of the professional to undertake a contract until it has been completed. This is a result of:

- the knowledge required to understand the subject-matter, which is often divorced from everyday life, being expressed in language that is unfamiliar to the public (that is, jargon);
- difficulties in relating the activities of the professional, such as a doctor performing a diagnosis, to the outcome for the client, which may have financial or ethical consequences that could not have been foreseen by the client;
- the variation between contracts, so that past performance can be only a guide to the future.

Yet the outcome of the professional's deliberations has an impact on the lay people contracting their services. Consequently, for these professional occupations to be viable, the practitioners must be able to engender *confidence* in themselves for potential clients, without relying on a description of their proposed plan of action. This can be achieved by a combination of personal attributes, such as trustworthiness, competence and ethical behaviour, and formal peer recognition. The last is important since a potential client who has not had a long association with the profession cannot establish whether the attributes are genuine or feigned, such as in the case of a person with no experience, but knowing the right words to use, masquerading as a software engineer.

If you consult a professional, you expect any advice given to be accurate and appropriate, any work resulting from the consultation to be carried out in a competent way, and the professional to take responsibility for the advice given. This determines the three cornerstones of professionalism:

- being *knowledgeable* about the field;
- being *competent* to practise;
- taking *responsibility* for one's actions.

As an example, consider the medical profession. If you went to see your GP with a skin rash, it is reasonable to expect them to be able to examine the problem, to suggest treatment, such as the regular application of some cream, and to provide you with the means of obtaining the cream together with appropriate instructions. The consultation should result in you feeling that the GP has examined your complaint, and has exercised the most up to date *knowledge* in recommending a solution. In this situation, you are prepared to hand over responsibility for sorting out the rash to someone whom you believe to be better placed than you, that is, someone more knowledgeable and experienced in treating skin rashes. Moreover, if applying the cream produced an even worse rash, you would expect to be able to go back to the same GP and have the situation corrected, although the situation may have shaken your confidence in their ability to sort out your problem! You would not expect the GP to insist that the prescription was correct or to refuse you further treatment.

Being *competent* to practise means that you are able to apply knowledge of the domain in making technical and ethical decisions. This is about showing that you can use your knowledge to good effect in a practical setting: for example, showing that you do not just understand the theory of networking, but can apply it appropriately in a given situation. Implicit in this is the need to have up to date knowledge of the subject, including the processes involved. In making competent decisions, it is assumed that all relevant information will be taken into account, all relevant options will be examined in sufficient detail to make a judgement, and the interests of all the people involved will be served adequately. This is quite a tall order. It is virtually impossible to say definitively that all the options have been considered. However, it is possible to make a judgement that enough relevant information, sufficient relevant options, and the

interests of a large enough group of people have been considered in making a decision.

The specialised nature of a professional's knowledge and competence invariably means that the services they render are valuable to the client. The fact that the remuneration for this valuable service is high when it is a success carries with it the complementary obligation of taking *responsibility* when things go wrong. The attitude of the professional is also an issue here, since, in addition to according clients their statutory rights in relation to faulty services, the professional is also expected to accept the responsibility for maintaining the public's confidence in the profession, for example in terms of their own practices in commissioning and paying for the services they require from others.

What does it mean to be a 'software professional'?

In answering this question, the cornerstones of professionalism – knowledgeability, competence and responsibility – are taken for granted. We shall look here at how three other aspects of professionalism contribute to the concept of a 'software professional', namely:

- being a member of a professional body;
- being clear in the use of language;
- being honest.

Being a member of a professional body

There are a number of professional bodies that claim to represent the software profession. In the UK, the most prominent include the BCS (British Computer Society) and IET (Institution of Engineering and Technology), while those based in North America include the IEEE (Institute of Electronic and Electrical Engineers) and ACM (Association for Computing Machinery).

One answer, therefore, to the question of what is meant by a 'software professional' is to accept that it means whatever organisations like the BCS and IET say it means! However, this meaning varies depending on the level of membership, and there are usually a number of routes to each level of membership. For example, one route to full membership of the BCS relies on a certain level of education within the computing field, and verification from existing members that you are competent. This highlights two of the cornerstones identified above: being knowledgeable and being competent. Note, however, that unlike for example the medical profession, there is no *requirement* for a software professional to be a member of a professional body.

There are also different ways of interpreting 'software professional'. It could be taken to mean a professional software engineer, that is, someone who is directly involved in creating a software artefact. However, it often also designates a professional who is involved in training users, or someone who is a professional user of software packages, and so on. The kinds of knowledge and competence that will be relevant to each type of software professional will vary, but the same professional issues are relevant, in varying degrees, to all types.

Being clear in the use of language

> All professions are conspiracies against the laity.
>
> (George Bernard Shaw, *The Doctor's Dilemma*, 1911)

All fields have their own specialist terminology or *jargon*, and software is no exception. Jargon is necessary because, within a discipline, it is important to differentiate between shades of meaning that are indistinguishable to the general public, and for which there are no generally recognised words. Jargon is useful and appropriate within

a discipline, and it is a professional's duty to use it consistently in dealing with other professionals. Outside the discipline, it is a professional's duty to ensure that jargon is not used to confuse and to make sure that any interaction, especially with customers, is clearly understood and free from abstruse language. These interactions occur through user manuals, in conversations with potential users or customers, in online help services or explanation facilities, and so on. The need for clarity is particularly acute when one is trying to understand what the user wants from a system during requirements analysis and specification.

Since software has become part of everyday life, and computers in the home have become commonplace, some of the technical terminology used in the field has become commonplace (indeed, the home computer market has developed jargon of its own!). However, this is no excuse for using jargon where plain language will do.

Being honest

It is important for software professionals to be honest and fair with themselves, their customers and their fellow professionals on a wide range of issues: from their own capabilities to the feasibility of achieving objectives.

There are obvious examples of unsubstantiated claims, such as claiming to be an expert on network systems when you don't know anything about them. If your knowledge of networking is more than five years old, it is questionable whether you should call yourself an expert. But given the rapidity with which technology changes, how dated must your knowledge be before claims of expertise become inappropriate? Keeping totally up to date can be a full-time occupation, leaving little or no time to apply the knowledge obtained!

The pressures brought to bear by users and potential customers are not necessarily helpful in this respect. For example, their perception that software is infinitely malleable and extensible can lead to expectations that are too high. It is not always desirable or possible to disillusion customers about the capabilities of software, but it is important to be as honest as possible.

The concerns considered so far relate to interactions with the customer or user. There are other areas that may indirectly affect the customer but have a more direct impact on fellow software professionals. One is the lack of sufficient up to date documentation for a software system; another is the use of ad hoc methods for software development. Both of these areas present additional problems and pressures for fellow software professionals. For example, if a software system to support a financial institution needs to be modified following a change in the law, it is important that the software is modified and that the documentation (technical and user-oriented) is brought up to date.

SAQ 4

Describe the three cornerstones of professionalism.

ANSWER.....

- Having up to date knowledge of the field that is relevant to the intended application.
- Being competent to make technical and non-technical decisions in situations arising from the application of that knowledge.
- Being prepared to take responsibility for the consequences of any advice given or action taken.

SAQ 5

Jargon may be used inadvertently and result in confusion; that is regrettable, and a professional should adopt practices to reduce this, for example by having some independent person read and comment on jargon in all documents. However, sometimes jargon may be used deliberately. Suggest one reason for an unprofessional person deliberately using jargon.

ANSWER.....

Jargon is one way to hide lack of knowledge or uncertainty. A professional should feel comfortable about saying when they are unsure, and be willing to go away and investigate further.

Exercise 1

Describe some of the problems attributed to software that you have encountered, and their consequences.

Solution.....

Here is the sort of answer you might have come up with.

There are frequent reports in daily papers of accidents in which software problems are suspected, if never proved. For example, in aviation, the railways or the nuclear industry, such as the crashes of the early Airbus 320s and of at least one Chinook helicopter. Other threats – such as those posed by the lack of privacy associated with email, sending information across the Web or using cellular telephones – are less dramatic than those associated with controlling a railway network, say, but can none the less have serious consequences.

Exercise 2

Look again at the lists of process-related human factors presented below Figure 1, and identify one more factor in each of the three categories: person-related; task-related; environment-related. Suggest how each chosen factor might interact with other factors.

Solution.....

There are many possible answers to this question. Here is one.

- Another *person-related factor* is the engineer's knowledge of the interfaces between the code module being developed and others. This factor depends on how effectively the system architecture has been communicated (another person-related factor).
- Another *task-related factor* is the complexity of the code. This factor depends on the engineer's experience of similar code (a person-related factor) and the tools available to help manage complexity (an environment-related factor).
- Another *environment-related factor* is the ease with which the representation used for the design specification can be transformed into code. That is, the programming language being used may not directly support the kinds of data structure used in the design. This factor is heavily influenced by the engineer's knowledge of the design representation (a person-related factor).

Exercise 3

Reflect on your understanding of the term *professional*, and list six characteristics you associate with the term. If possible discuss the term with colleagues and assess their views.

Solution

There are many characteristics, but some of those you might have listed are trustworthiness, competence, knowledge, ethical behaviour, specialist training, sound judgement, lack of bias, honesty, integrity, responsibility, and clarity.

Exercise 4

Is it important to you to be regarded as a professional? If it is, then why is it? If it isn't, then why isn't it?

Solution

There is no right or wrong answer to this exercise.

Reasons for wanting to be regarded as a professional may include increased earning capacity, marketability, greater respect from colleagues, trust (an individual known to operate under a consistent code of personal ethics is one who can be relied upon), security (employment as a professional is often perceived as offering more security) and comfort (this is admittedly subjective, but peace of mind may be an important benefit).

Reasons for it being unimportant to be seen as a professional are harder to find. Not wanting to appear cut-off as part of an elitist group might be a motivation for some.

Exercise 5

At the beginning of this section we described three flaws with the Therac-25 radiation treatment system. These could simply be blamed on an incompetent software engineer, but the quotation from Leveson and Turner suggests a broader reason for the flaws. Describe how these flaws might have been prevented had due regard to human factors and professionalism been taken.

Solution

We do not know enough about the context within which the software was developed, but flaws 1 and 2 are easy errors to make and the chances of these being carried through to production code could have been significantly reduced by recognising the possibility of human failure and introducing software inspections. However, since these are classic errors, we would have expected an experienced engineer not to make them, and management should have matched engineer experience (a person-related human factor) to the demands of this real-time problem (a task-related human factor). Equally the engineer should have appreciated the nature of the task and behaved professionally by not accepting the job, or by insisting on appropriate quality control processes such as inspections.

Flaw 3 is a system problem, and the need for both hardware and software interlocks should have been recognised as part of the safety analysis which should have been carried out had the company been behaving professionally. (Later models of the machinery did reintroduce the hardware interlocks, for safety reasons, recognising that safety is significantly enhanced by using a variety of complementary technologies.)

2.4 Summary of section

In this section you have been introduced to product-related and process-related human factors in relation to the software development process. You have learnt that the problems that beset the process of developing software have not been completely solved by the introduction of technological 'fixes', such as more methods or tools. This suggests that attention needs to be paid to process-related human factors.

Human beings bring many characteristics and skills to their work, and it is important to be able to focus on those characteristics and skills that are most influential in any particular context. In this section you have learnt a simple classification system for the process-related human factors important to software engineering, one that distinguishes between person-related, task-related and environment-related factors.

You have also learnt that, in general, being professional is associated with attributes such as knowledge, competence and responsibility. In the special case of being a software professional, we have looked at three other aspects of professionalism: being a member of a professional software body, the use of clear language and being honest.

3 Person-related human factors

In Section 2 you saw how process-related human factors can be categorised into those that are person-related, those that are task-related and those that are environment-related. In this section you will study person-related factors in more depth, and will see that there are two types of person-related factors.

Individual factors, in the context of software engineering, are those factors that arise directly from the characteristics and behaviours of the people involved in the software development process. In other words, they are concerned with the competence and motivation of *individual* software engineers, that is, how well they know how to perform their work and how well they actually perform it.

Commercial software, however, is not produced solely by individuals; typically, each individual is part of a group or team, which has a collective responsibility for the work. The group, in turn, is located in an organisational structure and must interact with managers and other technical groups within that structure, as well as with people and groups from outside. This social dimension of software engineering means that other person-related skills – skills concerned with social behaviour and communication – must be used to perform the work effectively, so we may distinguish between *individual* and *social* person-related factors.

3.1 Individual factors

Imagine that you are a software development project manager. You have a new project, and are recruiting engineers to the team. What individual factors would you look for in the candidates? Most people would emphasise the need for engineers with relevant knowledge and experience, and the ability and motivation to apply their skills effectively.

This suggests that the most important individual factors are:

- competence;
- motivation.

Competence

You have already seen competence as a mark of the professional in Section 2. Competence in a software engineer is a mixture of knowledge and experience with respect to a particular task or activity, and the ability to apply that knowledge and experience to the task. For any particular task, **domain-specific knowledge** – that is, knowledge specifically concerned with the particular task or type of application – is more important than general knowledge and intelligence.

Two points should be noted about competence.

- Competence in a domain is not solely the product of knowledge; an individual may have a number of qualifications in a given domain but lack the ability, experience or training to *apply* that knowledge effectively.
- Competence among individual engineers and within the development team is not a fixed asset, because as new people join the team, or their roles change, or new hardware and tools are provided, competence will increase or decrease. So, it must be actively maintained through support and training.

In software engineering, competence is required in the following domains, depending on the individual's role:

- application;
- system;
- analysis and design;
- development processes;
- development methods;
- development tools;
- development environment.

Lack of competence in the *application domain* leads to errors in requirements specification and design. For example, understanding of the real-world application of the software is crucial when one is trying to identify non-functional requirements, such as system response time and safety issues.

Competence in the *system domain* requires an understanding of how the software system integrates with other system components and of where the boundaries of functionality lie. This information is particularly important for design and coding. Lack of competence in the system domain can lead to errors in design and implementation, which may not become evident until integration and testing.

All software development methods involve significant analytical work, abstracting out the essential features of the problem while setting aside the irrelevant, choosing between alternatives and elaborating possible solutions in order to explore their properties and move towards an implementation. These generic *analysis and design skills* are essential, regardless of the specific development method chosen.

Knowledge of development processes is critical to being able to choose how the project should be organised. Should the project use a waterfall process, a spiral process, one of the agile processes, or something else? The answer will depend on the project, the time scales, the team, and the customer.

Competence in the *development methods domain* is essential, since the development method chosen determines the activities and representations that the engineer must work with: imagine being asked to produce a specification of software, using the object-oriented approach covered in this course, if you were only knowledgeable about methods involving the structured approach of data-flow diagrams and entity–relationship diagrams! A lack of experience with the development method used can lead to errors at each stage of the development process. So, adequate support and training for engineers must be provided, especially when new methods are being introduced.

Competence in the use of *development tools* is not always easy to acquire, as many of them are sophisticated and complex. They require a period of learning and practice if their undoubted benefits are to be maximised. If support and training are not forthcoming, errors are likely to be made at each stage of the development process.

Competence in the *development environment* demands knowledge and experience of the development hardware, operating system and programming language(s) being used. Again, a lack of competence in this domain is likely to lead to errors at each stage of the development process, but especially during implementation. Competence can be enhanced by providing a stable and well-integrated environment alongside support and training.

Motivation

Motivation is what arouses the individual to action and directs the nature of the action over time. Psychology has tended to distinguish between:

- biologically motivated behaviour, which is innate – for example, feeding and reproduction;
- socially motivated behaviour, which is learnt – for example, how to behave in a group.

In the context of software engineering, socially motivated behaviour has a significant influence on an individual's productivity and performance. Motivation is usually thought of as an internally determined characteristic, but as far as work is concerned it is determined both by certain characteristics of the individual and by certain characteristics of the job. High motivation is achieved by matching the characteristics of the individual with the characteristics of the job.

The primary characteristics that have been shown to determine an individual's satisfaction with their job and their motivation to perform well are:

These characteristics come from the Job Characteristics Model, proposed in Hackman and Oldham (1976, pp. 250–279).

- *growth-need strength*: a person's need to develop, learn new things and be stimulated;
- *social-need strength*: a person's need to interact meaningfully with other people.

For software engineers, those characteristics of a job that satisfy growth-need must be present at an appropriate level. The characteristics of a job that are necessary to satisfy growth-need are:

- *skill variety*: the degree to which a job requires a variety of different activities, and involves the use of a number of different skills and talents;
- *task identity*: the degree to which a job requires the completion of a whole and identifiable piece of work, that is doing a job from beginning to end with a visible outcome;
- *autonomy*: the degree to which a job provides the individual with the freedom, independence and discretion to schedule their work and to determine the procedures that will be used;
- *task significance*: the degree to which a job has a substantial impact on the lives and/or work of other people, whether in the immediate organization or in the external environment;
- *feedback*: the degree to which the employee receives information about the effectiveness of their performance.

Note that these last two characteristics of growth-need strength also have a social dimension.

If these characteristics are present, they create a sense of meaningfulness and responsibility in the engineer, and they should therefore be present at a level that matches the individual's growth-need in order to maintain his or her motivation.

A software engineer who lacks motivation is likely to make errors at every stage of the software development process. Motivation can be supported by ensuring that the role of the individual engineer is designed, and redesigned where necessary, to provide these motivating factors at an adequate level.

Social-need strength, which is a social factor, is discussed in the next subsection.

SAQ 6

A software engineer is developing the requirements specification for a control system for an oil refinery. How could the engineer's competence in the application domain be enhanced to ensure that non-functional requirements are captured?

ANSWER.....

By exposing the engineer to the way the software will actually be used: for example, by letting them see how the end-user works with the current software or manual system to perform the task.

SAQ 7

Suppose that the oil refinery software has reached the design stage. The software controls a potentially dangerous process and interacts with a number of mechanical devices to ensure that the system works safely. How can the engineer's system domain competence be enhanced to ensure that safety features are properly considered?

ANSWER.....

As becoming a petroleum engineer is impractical, the software engineer should interact effectively with those responsible for other system components: for example, the refinery system engineers should review the software design. In addition, because the system is safety-critical the software engineer should use formal methods, such as VDM (Vienna Development Method).

3.2 Social factors

The development of a software product inevitably requires people to work together. The scale of commercial software means that it must be produced by teams of engineers if it is to be delivered within realistic timescales. Within such a team, people from different disciplines, with different skills, are involved at various stages of the development process. The team must work with other individuals and teams in its own organisation, such as managers and system engineers. It must also work with people from outside the software development team, such as customers, users and auditors. Consequently, the individual engineer must be able to:

- work as part of a team;
- communicate effectively with others.

Working as part of a team

It is almost impossible to put two or more human beings together for long without a group developing. A group differs from a loose aggregate – for example, a crowd – by sharing certain aspects, perhaps with leaders and followers within a hierarchy of roles, status positions and rules. A group may or may not be a team, which is a collection of people linked together for a common purpose. Group hierarchies have a remarkably similar structure regardless of the kind of group in which they appear. So, for example, an amateur dramatic society and a software development team function in much the same way, because the people within each group have a learned understanding of *group dynamics*, and have a tendency to behave in ways that conform to this understanding. Such behaviours are socially motivated.

Consequently, when software engineers work together in developing a product, you can expect that these social behaviours will shape what happens within the team. You can also expect that deliberately organising the team in ways that match these learned behaviours will help the team to be more effective and productive. There are several aspects of team organisation that can be optimised, namely:

- team structure;

- team goals;
- team leadership;
- team building.

Team structure

In *Unit 13* you saw briefly some alternative ways of organising projects. Conway (1968, pp. 28–31) observed that the structure of a software system tends to mirror the structure of the team that produced it. This appears to be an inevitable phenomenon, and it can be beneficial if handled correctly. A hierarchical team structurewill produce a hierarchical system, with a high degree of modularity and structural integrity. Traditional wisdom is that these are desirable attributes of a software system, and that people prefer to organise themselves within such a structure because it makes roles, responsibilities and authority explicit. However, with software now being more distributed and executing in parallel, software systems are more loosely organised and this seems to legitimise looser and more democratic team organisations.

If an inappropriate system structure is arbitrarily imposed on a team, the team will devote a good deal of its energy to circumventing it. So it is important to involve the whole team at the design stage or earlier in deciding on the structure of the system and the team, as well as in the division of work.

Team goals

The development of a shared understanding of the group's objectives and strategies is crucial to project success. When team members cannot understand or identify with their goals, they will tend to work toward goals they have set for themselves. This can totally undermine the coordination of team activities and lead to expensive reworking and error correction. The most effective solution to this problem is to let the team have a say in the decision-making process, and in setting their short-term and long-term goals.

Team leadership

Certain features seem to be common to all human groups, and leadership is one of these features. In other words, a leader will tend to emerge in any group. Psychology tells us that leadership is related to power, and that a leader has power over a group because they control resources in relation to the group's dependencies. Leadership in a software development team often involves two types of resource: technical and managerial. The leadership role is often split to reflect this division. For example, a project manager may be appointed to deal with administrative matters, while a technical leader is chosen (or sometimes emerges by consensus) to coordinate the development work and keep the team moving toward its goals. Sometimes the technical leader is appointed, as in chief programmer teams.

Teams that do not have a clearly identified leader tend to be unsatisfactory for team members, as well as unproductive. However, teams that are subjected to an authoritarian regime do not prosper either; they are inclined to spend more time subverting the leader's power than developing software. It is generally found that a democratic leadership style raises both productivity and individual satisfaction.

To get good performance from the development team it is usually best if someone takes on the role of leader. If a team leader is officially appointed, it should be someone with enough technical competence to command the group's respect, and someone who can exercise authority in a way that does not cause resentment.

Team building

There are two aspects to team building:

1 getting the right people together;

2 helping them to become a team.

A team needs an appropriate balance of technical expertise across the relevant domains. There should also be an appropriate mixture of experienced and less experienced engineers; this means that junior staff can get proper supervision and training, and senior staff can get experience of management and mentoring. Mixing experienced and less experienced staff is also a way of maintaining and improving competence in the organisation.

When the team has been assembled, it may function simply as a collection of individuals. But the most productive teams move beyond this stage to one in which the team forms a group; individual actions are coordinated and focused on a shared goal. Helping the team to gel is mostly a matter of ensuring that every member contributes to, and identifies with, the team's goals, that the team can work in physical proximity, and that the team members' time and effort are not distributed across a number of projects.

For advice on helping a team to gel, see DeMarco and Lister (1987).

Communication

Communication is the life-blood of any development project. No team, however competent or motivated, can succeed without it. Whatever the medium, effective communication is crucial for the transfer of knowledge and the development of a shared understanding of goals and objectives.

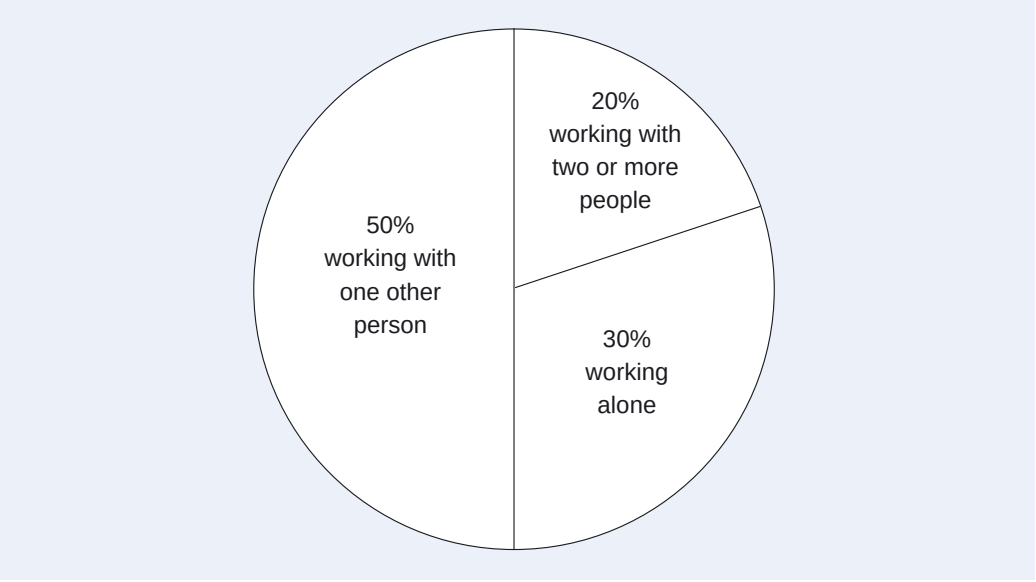

Figure 2 is based on McCue, G. (1978) 'Architecture design for program development', *IBM Systems Journal*, Vol. 17, No. 1.

Figure 2 How software engineers spend their time

Figure 2 shows that software engineers spend up to 70 per cent of their time communicating with other individuals. This suggests that effective communication (that is, easily established and well focused communication) can enhance productivity and the integrity of the product. This in turn means that *structures*, *paths* and *media* supporting effective communication must be built into the team structure, into its environment and into its activities.

Communication is most effective when the *structures* that support it are decentralised, that is, when information can circulate freely between team members and between teams. In situations where all information is given to one person for

distribution – for example, the team leader – bottlenecks develop and the flow of information is restricted. When a bottleneck develops, errors and reduced productivity are inevitably the result. For example, team members who continue to work from documents and specifications that have been superseded will find themselves either starting the work again or forcing what has already been done to meet new, and often conflicting, requirements.

Optimising communication structures can be difficult. For example, if everyone is informed about everything, the result can be as damaging as the bottleneck situation: people waste time processing information that is irrelevant to their work. Communication structures should reflect the team structure discussed above and be supported by the right communication media (see below).

Communication *paths* become degraded as the distance (physical and organisational) increases between the communicator and the listener. As the distance increases, it becomes increasingly likely that messages will be transformed from one representation of the information into another. For example, if a teleworker needs to discuss project progress with an absent manager, they may well have to write down their thoughts and fax, post or email them. This one-way communication without the opportunity to correct misunderstandings and engage in discussion, together with the transformation from a verbal to written representation, increases the risk of ambiguity and loss of content. This is rather like the problem of capturing requirements, seen in several earlier units. The content of any communication is further degraded by the omissions and interference it picks up as it passes from one person to another.

Communication suffers further when the parties involved have cultural and contextual differences. For example, when a customer talks to an engineer about software requirements, they have to establish a common language, that is, a shared understanding of real-world concepts, terminology and priorities. Even when an engineer talks to another engineer – two people who ostensibly have the advantage of at least a partially shared understanding – the communication process can still be problematic. A lot of the meaning in human communication is expressed through *non-verbal behaviour* or 'body language'. Without these reinforcing cues – for example, when using a telephone or email – the listener is less likely to receive the message as intended.

We conclude this discussion on communication paths by suggesting that important information should be transmitted as directly as possible, especially if it must cross a 'cultural' boundary separating one group from another, as in going from system engineers to software engineers.

In software projects, communication can be formal or informal, impersonal or interpersonal. The *media* supporting these different communication styles are not equally suited to all tasks.

- *Formal impersonal media,* such as documentation, schedules and data dictionaries, are useful for documenting decisions or providing common representations. However, they are not so useful for developing understanding or consensus, since the sender cannot guarantee that the documents have been read or understood. These methods are better suited to recording project progress than to achieving it.
- *Formal interpersonal media,* such as reviews and inspections, are useful for developing a shared understanding of the system and of short-term goals. However, to be most beneficial, they must be conducted in such a way as to allow problems to be aired without fear of blame or reprisals.
- *Informal interpersonal media,* such as group meetings and getting staff together, are the most effective way of keeping team members informed. The team's working environment should be structured to encourage this kind of communication.

- *Electronic media,* such as email, bulletin boards or computer conferences, blogs, and wikis can be useful, especially when team members or collaborating teams are physically distant from each other. However, an email maybe regarded by the sender as informal, yet treated as formal by the recipient, and this can lead to difficulties. The others may need moderation to remove inappropriate submissions.

SAQ 8

(a) What factors affect the functioning of a team?

(b) Identify two ways in which the functioning of a team can be supported.

ANSWER.....

(a) Team structure, team goals, team leadership and team building.

(b) Two out of the following:

- letting the team structure mirror the system structure;
- letting the team set its own goals;
- appointing an appropriate team leader;
- recruiting staff with an appropriate balance of skills.

SAQ 9

What type of communication media would be most suitable for:

(a) reviewing the detailed design?

(b) informing the team about changes to the requirements?

ANSWER.....

(a) Formal interpersonal media.

(b) Formal impersonal media.

Exercise 6

As project manager of an oil refinery software team, you are recruiting software engineers to work on the safety-critical parts of the system. These code modules are strictly partitioned and deeply embedded, interacting with mechanical and electrical safety devices rather than the end-user. In order to achieve certification, these modules must be developed using a formal method based on mathematics. Which domains of competence would you consider most important when deciding on whom to recruit?

For more information on formal methods, see http://en.wikipedia.org/wiki/Category:Formal_methods (Accessed 26 February 2008).

Solution.....

The most important domains in this instance would be the system, development methods and development tools domains.

Exercise 7

Which of the growth-need strength motivating characteristics might be lacking in the case of each of the members of the following development team?

- *Sara* is the team leader. She was promoted to this job a few months ago and this is the first project she has been in charge of. She is not too sure about some aspects of the job but cannot consult her own boss, who is out of the country for the foreseeable future supervising a much bigger project. Sara worries about keeping up with the project plan and finds it hard to delegate.

- *Dick* is the team's lead programmer. He has had a lot of experience and is used to making decisions and keeping the work moving along. At the moment, he has to get Sara's agreement before he can allocate work or revise schedules.
- *Harriet* joined the team as a junior programmer six months ago. She has always done her work promptly and competently, but she still tends to be given only small implementation tasks, such as documenting and tidying up Dick's code.
- *Dennis* works part-time for the team as an independent tester. When modules are completed, he runs them through an automated test harness and passes the results to his own boss, who is not part of the team. He spends the rest of his time doing the same thing for other teams.

Solution

- Sara's main problem is probably lack of *feedback*. Her boss's absence means she is not getting an objective appraisal of her effectiveness.
- Dick suffers from a lack of *autonomy*. He needs Sara to acknowledge that he is sufficiently competent to use his discretion in planning the group's work.
- Harriet's job is not providing much in the way of *task identity* and *task significance*. She needs to be allowed to take on more responsibility. Dick should assign her some particular area of work, however small, and then monitor her progress.
- Dennis has all of Harriet's problems plus a distinct lack of *skill variety*. If he has the aptitude, he should be trained to analyse and report the test results.

Exercise 8

Suggest areas where communications may not be optimal in the scenario described in Exercise 7.

Solution

- Sara cannot consult her own boss so there is no communication at all at this level.
- Sara finds it difficult to delegate, which may indicate an unwillingness to divulge information.
- Harriet may expect her work to be fully specified and may not see the need for communication.
- Dennis may see his main responsibility being to his own boss and not to the development team.

3.3 Summary of section

Software development is carried out by individuals working together in teams. These individuals have different technical skills and responsibilities. In this section you have learnt that it is important that software engineers have competence in a number of domains, since it is at domain boundaries and interfaces that misunderstandings arise and errors are introduced into the product. In addition, individuals must be well motivated if they are to perform effectively.

The need to work together as a team means that the social dimensions of work are influential in determining the success or failure of a software project. However, human beings are naturally equipped to work in this way, and can benefit from efforts to tailor their working environment and practices to match their abilities. The media by which communication takes place can both help and hinder the flow of information.

4 Task- and environment-related human factors

In Section 3 you studied those process-related human factors classified as person-related. In this section we shall look in more detail at the other two process-related human factors, described as task-related and environment-related. We shall also look at automation and the use of tools.

4.1 Task-related factors

Task-related factors arise from the particular task undertaken, and have an indirect effect on the engineer's motivation and ability to apply his or her knowledge and experience. There are three aspects of software development tasks that exert a particular influence on human performance:

- complexity;
- familiarity;
- variability.

Task complexity

The complexity of a task determines the extent and depth to which an engineer must reason about the task, and the representational forms that are appropriate. The human brain is adapted to cope with a limited number of things at a time, but it has particular problems with multidimensional data. This is why development methods and their associated tools are useful: they allow the user to separate out the different dimensions of the problem and so reduce the overall complexity of the task.

Very complex tasks take a long time to complete and are more likely to introduce errors. The proper use of tool-supported graphical notations can provide an enormous advantage. The graphical notations are generally easier to absorb and understand, and they facilitate the checking of consistency within and across different views of the same task.

Task familiarity

If the engineer has performed a similar task before, they can reuse some or all of the previous problem-solving strategy (provided it was successful!). This is known as *learning by analogy*, and forms an important part of the human problem-solving repertoire. If there is no similarity between a new task and what has gone before, or if domain-specific details are particularly novel, the engineer must develop a new strategy.

Obviously, the more familiar the task, the faster and less error-prone the performance of that task will be. Where the task is very unfamiliar, learning materials and support must be available, and adequate time must be allocated to the learning process.

Task variability

Task variability concerns the extent to which a task is repetitive. In view of what has been noted about complex and unfamiliar tasks, you might expect repetitive tasks to be completed quickly and reliably, and to a certain extent this is the case. However, tasks of this kind are subject to errors that arise because human beings have, to a

large extent, automated their performance and 'switched off' their higher processing functions. This is called *skill-based behaviour*. When engineers are functioning at this level, they are easily triggered into producing inappropriate responses if the task presents a stimulus similar to the one expected.

Skill-based errors can be trivial, but they are time-consuming to trace and correct; for example, a simple syntax error in a program can be surprisingly difficult to spot. For tasks that are highly repetitive, automation is likely to yield benefits. (Automation is considered later in this section.)

SAQ 10

Which task-related factors might affect performance of the following tasks:

(a) modelling the time-dependent characteristics of a large real-time system?

(b) formatting a program listing?

ANSWER.....

(a) Task complexity – the task complexity is likely to be too high.

(b) Task variability – there is likely to be a lack of task variability.

4.2 Environment-related factors

The environment that surrounds the software development process has a variety of components. There is the *physical space* in which the work is done – in this context, usually an office. Then there are the *methods* and *tools* that support the software engineer. All of these components can compromise productivity and increase the likelihood of errors if they are less than optimal.

Physical workspace

Human beings are amazingly adaptable, and seem able to function in many different environments. However, certain aspects of the working environment can have a negative affect on performance – not to say mental and physical health – if they do not meet certain requirements. Consequently, minimum legal working conditions – for example, minimum space requirements – are specified in a variety of health and safety directives.

However, apart from ensuring that engineers have enough physical space to locate both themselves and their work resources in some reasonable degree of comfort, there are other environmental factors – apart from those covered by health and safety directives – that can affect performance. McCue (1978) studied the office environment for IBM and concluded that the most important factors are:

- privacy;
- outside awareness;
- personalisation.

Privacy

Many software engineers work in open-plan offices. Although this provides some benefits if the people grouped together are working on related activities, it introduces major disadvantages in the form of noise and disruption. Many software engineering activities require high levels of attention and concentration. When they are interrupted by extraneous noise and activity, errors will almost certainly be introduced. Productivity will also be lowered if people are forced to break off from their tasks to attend to

telephone calls and enquiries. If it is not possible to screen or partition individual workspaces to give adequate privacy, McCue suggests that there should be space set aside in which people can work without interruption.

Outside awareness

In McCue's study, 'outside awareness' is defined as working in natural light with a view of the outside environment, that is, in a room with windows. However, this concept can be extended to mean acquiring an awareness of the group environment while minimising its intrusiveness. A well designed, open-plan environment, which also caters for the needs of privacy, can help to satisfy this requirement.

Personalisation

This is the ability to arrange the workspace to suit individual work practices and preferences. This requirement includes being able to vary heat, light and ventilation, as well as being able to alter the arrangement of the furniture and personalise the computer, perhaps even influencing the choice of computer to be used.

4.3 Methods and tools

You will have noted throughout this course that development methods and tools can be of immense value, as they allow us to manage the complexity and scale of the software development process and to support or automate those activities that are repetitive and well understood.

However, as was noted earlier in this section, methods and tools constitute two domains where competence on the part of the software engineer is important. Thus the value of any tool or method is dictated not only by its suitability for the development task and by whether it matches the requirements of engineers, but also by the skills of its user. From a human factors perspective, there are three primary issues to be considered when choosing methods and tools:

- ease of learning;
- ease of use;
- availability of training and support.

Ease of learning

The easier a tool or method is to learn, the more likely it is that engineers will achieve a sufficient level of competence in its use. *Ease of learning* ensures that the cost of developing this competence – the so-called 'learning curve' – is not too great. A tool may be easy to learn because it has very few functions, or because it is well structured and appears simple even though it is functionally rich.

Ease of use

Ease of use ensures that engineers are motivated to apply their competence and maintain it, rather than let their use of the tool lapse. Note that the usability issues covered in many places throughout this course are relevant to software tools. In particular, there may be trade-offs between ease of use and ease of learning, since a good tool would include short-cuts and other support for a proficient and regular user working at great speed, while beginners would start with other simpler ways of working and have to make a transition later on to use the experts' methods.

Availability of training and support

Many development activities are complex and difficult, and it is therefore to be expected that many of the methods and tools that support these activities will reflect this complexity. However, this need not be a bar to their use, provided that adequate provision is made to support engineers as they learn to use these methods and tools. Training and support are particularly important when new tools and methods are introduced, and new tools and methods should not be introduced unless the benefits of their introduction outweigh their costs. Support may be as simple as ensuring that sufficient documentation exists and is easily available (such as access to a helpdesk or FAQs), or as complex as developing training programmes and materials.

SAQ 11

Which environmental factors can affect the speed and quality of the performance of a software engineer?

ANSWER.....

Physical workspace, methods and tools.

SAQ 12

What are the advantages and disadvantages of an open-plan office as an environment for software development?

ANSWER.....

- *Advantages:* development team members can be located in close physical proximity and so benefit from opportunities for informal contact and communication.
- *Disadvantages:* unless designed with care, this environment does not provide sufficient privacy for tasks that require high levels of concentration.

4.4 Tools and automation

The role of software development tools

As this unit has progressed, you have learnt that, in a typical software development project, the bulk of the costs and the errors can be attributed to the activities of the people involved in the process. These human factors manifest themselves at a variety of levels – from the individual engineers, the organisation in which they work, and the market for their products, through to the society in which the product must function. Human factors involve many physical, mental and social characteristics, and pervade the software development process from the commissioning of software to the maintenance and use of the installed system.

At first sight, the full automation of the development process – at least of the core activities of design, implementation and testing – may seem an appealing solution to the problem of human cost and error, as it appears to offer an opportunity to replace fallible human beings with reliable tools. If we were content to settle for the kind of software we have now, large-scale automation might be a viable proposition to produce more of the same or similar – but it is primarily for its potential to evolve and adapt and solve new problems that we value software. When new roles and uses for

software are required, human creativity remains the only means of achieving them at the current time and probably for the foreseeable future.

Cartoon by Giles Reed

Figure 3 Is it just a cost-cutting exercise?

Consequently, any attempt to improve the development process must acknowledge and harness the skills that people bring to the process. However, automation is a viable option for some development tasks. Even where humans are still required and relied on, they can still be supported by tools playing a more clerical role in the overall process. The remainder of this subsection considers how and when to automate.

Computer-aided software engineering (CASE)

Although software engineering is often concerned with automating the work of others, the use of automated tools within the discipline itself is a relatively recent development, with the exception of coding tools like compilers that have been in use for very many years. However, CASE tools are now available for a number of software engineering activities. You have been using two of these on this course – a program development tool for Java (the IDE) and a design tool for UML (the modelling tool).

Some people take the view that automation is best applied to implementation and testing, but is not as appropriate for specification and design. The majority of tools manage, configure and trace items in some kind of repository, but few are associated with creativity or problem-solving. Until the mechanisms that underlie many software development tasks are more fully understood, we can expect that software engineering will remain a predominantly human-centred process.

Table 1 shows the relative capabilities of humans and machines that are relevant to software engineering activities.

Table 1 A comparison of human and machine capabilities

Humans generally better at	Machines generally better at
Detecting stimuli in noisy background.	Counting or measuring physical properties.
Recognising constant patterns in varying situations.	Storing quantities of coded information accurately.
Sensing unusual and unexpected events.	Monitoring pre-specified events, especially infrequent events.
Remembering principles and strategy.	Making rapid and consistent responses to input signals.
Retrieving pertinent details without a priori connection.	Recalling quantities of detailed information accurately.
Drawing upon experience and adapting decisions to situations.	Processing quantitative data in pre-specified ways.
Selecting alternatives if original approach fails.	Maintaining performance over extended periods.
Reasoning inductively: generalising from observations.	Reasoning deductively: inferring from a general principle.
Acting in unanticipated emergencies and novel situations.	Performing repetitive pre-programmed actions reliably.
Applying principles to solve varied problems.	Maintaining operations under heavy information load.
Concentrating on important tasks when overload occurs.	Performing several tasks simultaneously.
Making subjective evaluations and developing new solutions.	

Exercise 9

In Section 3 you learnt that human factors are often interactive. Which task-related factor from this section will be most heavily affected by a lack of privacy in the development environment?

Solution .....

The more complex the task, the more concentration is needed to accomplish it, so the answer is *task complexity*.

Exercise 10

By looking back through Sections 3 and 4 as necessary, draw a tree diagram showing a taxonomy of process-related human factors.

Solution

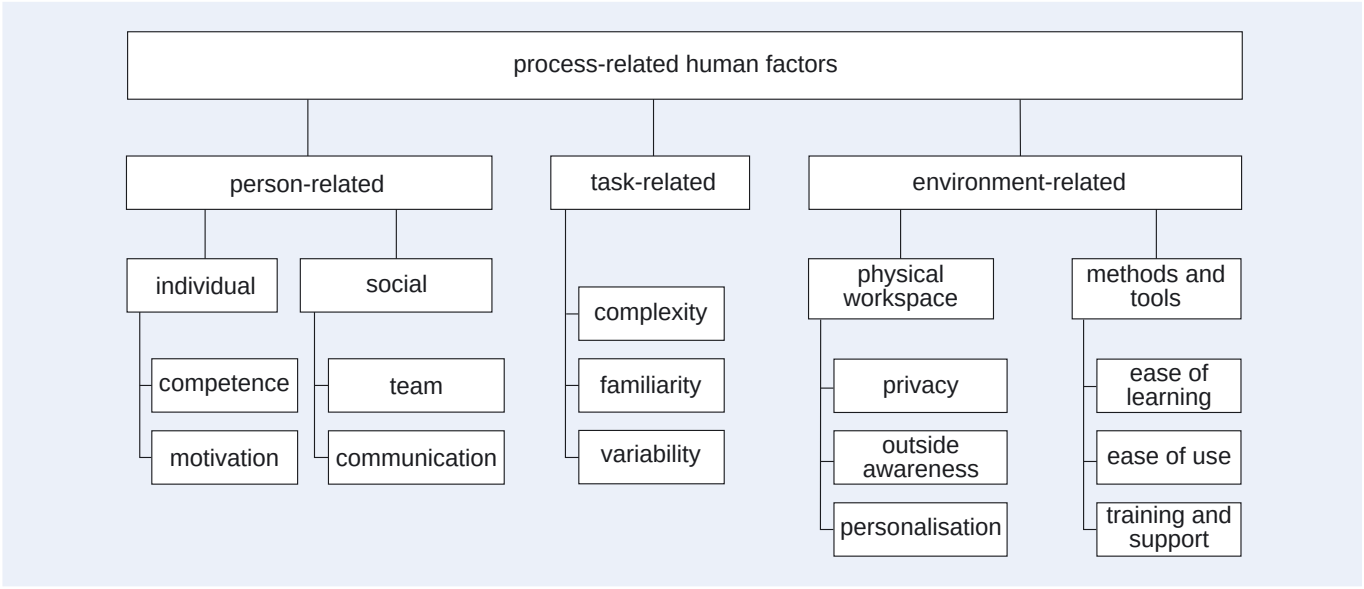

Figure 4 A tree diagram showing a taxonomy of process-related human factors

Exercise 11

Use Table 1 to decide which of the following activities in the software development process are suitable for automation and which would be best done by human beings:

(a) managing a clerical procedure;

(b) recalling data;

(c) evaluating a solution;

(d) creating a new process;

(e) storing data;

(f) processing data.

Solution

The implication of the comparisons in Table 1 is that activities (a), (c) and (d) would be best done by human beings, whereas activities (b), (e) and (f) would be best automated, making use of the computer's fast, reliable and accurate memory combined with its extremely fast feedback mechanisms.

4.5 Summary of section

The difficulty of developing software is in part associated with the types of task that occur frequently in software development. Environmental factors, including both the physical space in which software development is undertaken and the tools and methods that are used to support the development process, can also reduce productivity and accuracy. Although software development still relies heavily on the creativity and problem-solving abilities of human beings, there are some tasks that are suitable for automation.

5 Ethical and legal issues

In this section we shall analyse ethical and legal issues relevant to professionals involved in software development.

5.1 Ethical decision making

Recognising an ethical issue

The terms *ethics* and *morality* have many shades of meaning. In the context of this course, **ethical decision making** is about choosing between behaviour that is morally 'right' and that which is morally 'wrong'; **morality** is about the degree of conformity to a set of principles that determine 'rightness'. The trouble with these two definitions is that there is no accepted absolute definition of 'right' or 'wrong', and what is morally 'right' for one person in one set of circumstances is not necessarily morally 'right' for another person in the same set of circumstances (or indeed, for the same person in different circumstances).

There are at least three areas that influence the set of moral principles by which a person behaves.

- Individual or personal standards of behaviour, which arise from the person's experience, religion, family tradition or philosophy of life. The existence of such personal standards makes ethical decision making a very sensitive area.
- Social or cultural standards of behaviour, some of which may be enshrined in secular or religious law, and which arise from historical and religious traditions.
- Professional codes of conduct, which also arise in some sense from a culture: in this case the software professional's culture.

As you will see in Section 6, professional bodies issue codes of conduct and practice to their members.

An **ethical decision** is a decision which may result in *a living thing, including its environment, being adversely affected – emotionally, practically or morally*. Informally, an *ethical decision* is one that might 'make you feel uncomfortable'. From a professional standpoint, the important point is that the consequences of any such decisions are carefully thought through, and are not avoided; ignoring a situation or avoiding the problem is a decision in itself.

A whole variety of situations can give rise to ethical concerns. These situations may arise within the software development process (see Example 1) or may arise from wider considerations, such as the use to which the software being developed will be put (see Example 2). Whatever the situation, you can guarantee that there will be many things to consider. The examples presented here necessarily avoid some of the complications that would arise in a real-life situation, but they serve as useful illustrations none the less.

Example 1

Imagine that you are part of a team developing software to control a chemical plant. This software is safety-critical; if the chemical plant goes wrong, there could be a serious leak of lethal chemicals, which would affect the local town and lead to environmental pollution. You joined the team after some initial

decisions had been made, including the choice of development approach and development environment. The approach chosen involves no formal methods and no rigorous proof, as would be advised as best practice for the development of safety-critical systems. You draw this to the attention of your project manager, suggesting that the use of a more formal approach would be appropriate, but you are given a very brusque reply, which you feel does not fully address your concerns. What should you do?

- Drop your concern and get on with the job?
- Take the issue higher in the organisation?
- Take the issue to the client?
- Use a formal approach in areas where you feel that it is appropriate, no matter what the project manager may say?

Example 2

Imagine that you are employed as a project leader for the development of a new computer system, which has as one of its goals the replacement of as many manual workers in the client's company as is possible. You also learn that the company has no intention of helping these workers find new employment. What should you do?

- Get on with the job and ignore this aspect, as it is not your concern?
- Raise the issue with your bosses?
- Raise the issue with the client?

Behaving ethically involves being prepared to take on the responsibility for your actions; responsibility has a great influence on questions relating to ethical behaviour and, as you saw earlier in this unit, is a cornerstone of professionalism.

An action that is illegal may be considered unethical by default, assuming that you believe in living by the law of the land, but the reverse is not necessarily true. For example, hacking into the computer system of a bank and siphoning off money into your own account is illegal, and also represents an unethical use of your knowledge and expertise. But claiming to be competent in a specific area, such as point-of-sales technology, when in fact you have never set eyes on such equipment and don't know the first thing about it, may not be illegal, but it is certainly unethical ... or is it? You might even view some laws as being unethical, such as laws that suppress minorities.

Tackling ethical decision making

Once you have recognised that a situation involves an ethical decision, what should you do? Some issues, such as questions relating to equal opportunities or access to personal data, straddle the ethical and legal boundary. Some areas are covered by legislation; others are left to the individual's conscience. A professional approach includes a careful consideration of the possible courses of action and their consequences, remembering that ignoring or avoiding an issue is in itself a decision that will have consequences. Issues to be considered include your responsibility to users, to clients, to your colleagues both within your organisation and across the software profession, and to the wider community. Each situation should be judged on its own terms, and the chosen resolution will depend on your own set of personal beliefs as well as any responsibilities mentioned above.

When faced with a problem that involves an ethical decision, it can be difficult to take an objective view of the situation. The following eight-step procedure for coping with ethical decisions is an amalgam of methods found in the literature. This approach is not the only way of solving these problems, but it is one way to come to terms with a difficult situation. The procedure assumes that the decision involved is indeed an ethical one, and that the issue itself is fully understood.

1 Is the issue a symptom of a wider concern? If it is, re-evaluate the situation and identify the wider issue. If it is possible to tackle this wider issue, then do so, possibly in parallel with the current issue.

2 Is the situation governed by law or organisational rules? If so, the resolution may be dictated in whole or in part by reference to the laws of the country in which you are working, or according to the guidelines or rules of your client or any other organisation involved. You should be on the lookout for implications of this sort throughout the process.

3 Identify any party (for example, living creature, community, organisation or environment) that will be affected emotionally, practically or morally by a resolution of the problem.

4 Identify the main options and their justifications, and think about the expected outcomes of each option from the viewpoint of each of the parties identified in step 3. This process allows you to consider all sides of the argument, and to appreciate the implications of the action you finally decide to take.

5 Eliminate options that are 'illegal' as described in step 2.

6 For each remaining option, ask questions exploring the negative aspects of the expected outcome. For example, will this option lead to the privacy of certain users being compromised?

7 For each remaining option, ask questions exploring the neutral or positive aspects of the expected outcome, so as to make the 'best' choice.

8 Select a course of action, based on the reasoning above, which provides the greatest satisfaction to the greatest number of people, and which does not violate anyone's rights or minority interests.

In particular, these steps are based on ideas in Conger (1994), Langford (1995) and Kallman and Grillo (1996).

SAQ 13

Suppose a software engineer is part of a support team for a small software house. One of her clients, a large money broker, calls her first thing in the morning with a problem: last night's overnight run to produce a report of the day's dealings failed, and the system needs to be reset for the start of trading. Resetting the system for today's trading will delete all data from the previous day. The reports generated by the overnight run are required in order to complete the deals swiftly and efficiently. Without these reports, and due to the subsequent complications of completing deals through a different recording system, the client will lose money. However, there is also a risk that the client will lose money if the system is not running to support today's trading. The engineer's manager tells her that supporting today's dealings should take priority. Any losses incurred by the client due to system downtime will be charged to the software house, and so the engineer must get the system running under any circumstances. On arriving at the client's premises, the engineer is told by the client that under no circumstances should the overnight run be lost. No provision was made in the original system set-up to cope with the failure of an overnight run. What should the engineer do?

ANSWER.....

The engineer involved is facing an uncomfortable ethical decision, which may have serious implications for her personally, for her employer, for the client, and for the

client's customers. To help her make a decision, she could make use of the eight-step procedure as follows.

1 *Is the issue a symptom of a wider concern?* Yes, it is, in that, if these reports are so important, steps should have been taken to allow for this possibility in the original design. This issue can be tackled later, but the urgency of the situation does not allow an immediate consideration of the wider issue.

2 *Is the situation governed by law or organisational rules?* It is not obvious from the description that any laws or rules have been broken. However, deals are legally binding. So, if there is a danger that the deals will not be completed, there may be a legal aspect to any decision made.

3 *Identify any party affected*. The people primarily concerned are the client, the engineer, and the engineer's employer. Secondarily, the client's clients and the client's employees may also be affected if the company loses money on today's or yesterday's dealings.

4 *Identify the main options, their justifications, and their expected outcomes*. Three actions open to the engineer are as follows.

   (a) Redo the overnight run immediately, holding up the current day's trading. This option is supported by the client's comment that the overnight run must not be lost. The outcome of this course of action should be that the reports are generated, but the trading for the current day will be held up.

   (b) Forget the overnight run and initialise the system for the current day. This option is supported by the employer's comment that the current day's trading must not be disrupted. The expected outcome of this option is that the reports will be lost, but the current day's trading will not be disrupted. Note that the loss of the reports will result in difficulties in completing yesterday's deals, but the deals will be completed.

   (c) Backup the data for the overnight run, and try to run it on a different machine, or later on, without disrupting the current day's work. The expected outcome of this option is that the reports will be generated, as required, and minimal disruption of the current day's trading will occur.

5 *Eliminate options that are 'illegal'*. None of the options is illegal in terms of the law and, assuming that there is no rule within the client's organisation forbidding the copying of data, then none appears to contravene organisational rules.

6 *Ask negative questions about the expected outcomes of each option*. The questions to be considered should include general queries, such as: How much goodwill from the client would be lost if this option were chosen?

   (a) How much money will be lost owing to the delay in today's trading? What repercussions may there be if the engineer ignores the employer's view that the current day's trading must not be disrupted?

   (b) How much money will be lost if no reports are generated from yesterday's trading? What extra resources will be needed to complete the deals?

   (c) How safe is the backup procedure? What are the risks of running the reports at a later time, or on a different machine?

7 Ask neutral or positive questions about the expected outcomes of each option.

   (a) How much money will be saved by not losing the reports on yesterday's deals? How much (if any) recognition will the engineer get for following the client's wishes?

   (b) How much money will be made if today's trading is started immediately? How much (if any) recognition will the programmer get for following the employer's wishes?

   (c) How long will the backup take, before today's trading can begin?

8 *Select a course of action.* In our opinion, the course of action least likely to cause anyone concern, and to satisfy as many people as possible, is the compromise position: option (c). There are risks associated with backing up and generating the reports from a different machine, but if it is successful all parties will be happy.

SAQ 14

How would you characterise an ethical decision?

ANSWER.....

Ethical decisions involve deciding between what is morally 'right' and what is morally 'wrong'. An ethical decision is one that may cause a living thing to be adversely affected emotionally, practically or morally. Informally, it is one that makes you feel uncomfortable.

5.2 Legal issues

The previous subsection made it clear that ethical decision making must always be carried out with reference to the law. Computer law is a very complicated area, and we cannot hope to cover it in any depth in this course. However, when dealing with software and software-related issues, you have legal responsibilities and legal rights. Legal systems and the laws they uphold vary from country to country, and also change over time. The topics covered here are intended to be as general as possible, but we shall also draw on specific laws and directives to illustrate the points made. We will particularly be concerned with legal issues involving:

- contracts;
- intellectual property rights;
- data protection;
- computer misuse;
- health and safety.

Contractual issues

Legal contracts set out the parameters of an agreement, so that all parties understand their responsibilities. You may have come into contact with a number of different types of contract: contracts of employment, insurance contracts, consultancy contracts, contracts to supply software, and so on. Anything that contravenes these contracts is illegal and can result in legal action being taken. Thus, it is important that all parties to a contract understand what is required of them. This may sound fairly obvious, but there are some issues, such as confidentiality and intellectual property rights, which may not be considered during the drawing up of contracts. Often, too, the implications of what is stated in an agreement are not fully realised by those party to it.

For example, if you are an employee, you will have signed a contract of employment, and you would expect it to include something about your terms of employment, such as your holiday entitlement, hours of working, the duties expected of you, and so on. Your employer, in turn, will enter into contracts with clients, which set out the terms of the agreements, including the responsibilities of both the client and your employer. Among these terms may well be a statement of confidentiality, and you will be bound by that statement as an employee of the company. What happens after the completion of the contract may also be determined.

Note that there are some inalienable rights related to employment and consumer protection that cannot be overridden by contracts, however phrased.

Ownership of software

The ownership of software is a thorny legal issue, one that gives rise to many situations which are difficult to disentangle. The principal reason for the difficulty is the intangible nature of software. The kinds of problems that can occur when considering the ownership of software are indicated by the following examples.

- If your company develops a software system for a client, can the client modify the system and sell it to other companies? Can your company produce modified versions and sell those to other customers?
- If you, as a member of the project team, gain expertise through working on the system, and then move to a new company, can you use this expertise to develop a system for the new company? What use can be made of design ideas, sketches, prototypes and so on produced in the development of the original system?
- If you come up with a novel application and discuss it with friends and colleagues via an electronic news group, would you then be able to gain a patent to develop the idea and market the product? What if someone else takes your idea and develops a commercial product from it? What rights might your employer have over any wealth generated by your idea?

The contracts between you, your organisation and the client may cover some of these eventualities, in which case the terms of the contract will dictate what can and cannot be reused, any time delays required between the original development and further exploitation, and the ownership of ideas and creations. However, it is not uncommon for these issues to be overlooked or misunderstood when contracts are being drawn up.

As technology and software development move forward, other issues arise. For example, with self-correcting code, neural networks, genetic algorithms and other software that shows a level of 'intelligence', who is responsible for the software? Who owns the software? Is it in fact 'owned' by anyone?

Copyright and patents

The law can be used to protect ideas in two ways: through the **law of patents** and through **copyright law**. Together these are known as *intellectual property rights* (IPR). Protecting ideas is generally believed to be in the general interest of society, since without that protection innovation and invention might not be forthcoming. This view is supported by the national governments of the world's dominant nations, who campaign for IPR protection and enforcement internationally. However, not everybody agrees with this position, especially in relation to software. For example, the *Free Software Foundation* is an influential organisation fighting against software patents in particular, and IPR protection in general.

Patents are used to protect *inventions* that are new or involve a novel process and are capable of industrial application; the subject of the patent must not, however, be 'obvious' and must not have been published before. For example, the data compression technology used by Stac Electronics in their disc-compression product *Stacker* was considered non-obvious and patentable; the patents were subsequently upheld in 1994 in a court fight between Stac and Microsoft. A patent gives the owner a monopoly to exploit the invention for a number of years, and provides protection for the idea behind an invention. Software patents are a controversial and rapidly evolving subject.

Copyright, on the other hand, is automatic (that is, you do not have take any special action such as paying to have the subject registered) and it protects the *expression* of

an idea, not the idea itself. Copyright law is often used to protect software, but it has proved difficult to decide what constitutes the expression of an idea. Does it include the structure and organisation of the software? If you take the same system and translate it into a different programming language, the expression has changed but the idea is still intact. So, has copyright been infringed? The functionality of a system embodies ideas about how best to support people in their work. So, do systems that provide similar functionality infringe copyright? This latter notion seems absurd and surely cannot be the case: for example, how many word-processing packages exist with virtually identical functionality? These issues pose real problems for both software professionals and lawyers, and definitive answers to these problems are difficult to produce; cases brought to court can take weeks to consider. However, over the past 20 years or so, court cases have set a variety of precedents, and the following four examples, adapted from a book by David Bainbridge (1994) *Software Copyright Law,* illustrate some of the judgements given.

1 *Digital Communications Associates (DCA) v Softklone Distributing Corporation (SDC).*

The plaintiff (*DCA*) designed a screen display for a communications program which showed a list of commands with the first two letters highlighted and in block capitals; the user selected a command by entering its first two letters. The defendant (*SDC*) developed another communications program with a similar display. The court upheld protection of the screen display, saying that the *idea* was the concept of such a screen (and therefore not copyrightable), and the *expression* (that is, the use of highlighting and capital letters, and the organisation of the command) was the means to communicate the idea (and hence was protected by copyright).

2 *Broderbund Software (BS) v Unison World (UW).*

This case also concerned screen displays. In this instance, it was argued by the defendant (*UW*) that there was only one way to structure the screens and input, thus they were part of the idea, and not simply expression. In fact, other versions of the screens were produced as evidence, and it was ruled that the screen displays were indeed part of the expression used by *BS*, and thus copyrightable.

3 *Ibcos Computers Ltd v Barclays Mercantile Highland Finance Ltd (UK).*

A software engineer had worked on an accounts package for the plaintiff (*Ibcos*), and subsequently marketed a competing accounts package for the defendant (*Barclays Mercantile*). Copyright was held to subsist in the individual program and in the entire software package as a compilation. This case showed that, as well as individual computer programs being protected by copyright, the way they are linked together (structured) may, in some cases, also be protected. In other words, depending on the skill and judgement involved in selecting and arranging the individual programs, copying structural and design features may infringe copyright.

4 *Lotus Development Corporation v Paperback Software International.*

This case showed that overall organisation and structure, the content and structure of commands, and the user interface (choice of words or symbols) are protected by copyright, but the judge in this case said that it does not follow automatically that every expression of an idea is protected by copyright. He listed four things that must be considered:

- originality – the expression must originate from the author;
- functionality – if the expression simply embodies functional elements of an idea, it is not copyrightable;
- obviousness – if the expression is inseparable from the idea, it is not protected;

- merger – if the particular expression is one of a quite limited number of expressions, then it is not copyrightable.

Trademarks and service marks

The names of products and services are themselves protected as valuable property, and existing marks should not be used for other products or services in the same area of business or economic activity. Trademarks and service marks are registered. In documents such as this one, a trademark or service mark must be acknowledged, as seen below.

Free and open-source software

In the 1980s, a programmer called Richard Stallman established the Free Software Foundation (FSF), a movement that campaigns to prevent the commercial exploitation of software and to prevent restrictions being placed on the use of software. His aim was to provide software that, once bought, could be modified and distributed freely to others. The original piece of free software was a UNIX-like operating system called GNU, which, recursively, stands for 'GNU's Not UNIX'.

UNIX is a registered trademark of The Open Group.

The term **freeware** is now used to refer to a piece of software that is given away free to users, although the copyright remains with the originator and the original software cannot be distributed by anyone else. **Open-source software** is similar to freeware, except that it is maintained and upgraded openly by a community of expert users. If software is free to its users and the originator signs away the copyright, it is referred to as **public-domain software**. This means that users can modify the software, redistribute it or incorporate it into their own work; that is, they can do whatever they like with it!

Shareware, on the other hand, is software that is intended to be trialled for a limited period (often 30 days). Users are expected to register and pay any appropriate fee after this time has elapsed. Often the onus of registering after the time is up is left to the users, although other tactics, such as timed use or printing help files backwards, are sometimes employed to persuade people to register. Shareware represents the full functionality of the software, and is basically a way of marketing the product.

Data protection

The power of computers to store, retrieve and manipulate data quickly, accurately and efficiently has resulted in a number of advantages, such as facilitating information exchange for like-minded people (through mailing lists, for example), preventing fraud or other crime by corroborating information to check that people are indeed who they say they are, and speeding up the payment of bills. Information about each of us is held by a wide variety and a large number of organisations, and the advantages of this have now been tempered by the recognition of the potential dangers of misrepresentation and the threat to our privacy. If the information held is inaccurate or exaggerated this can have repercussions in terms of the credit we can gain, the jobs we may hold, and so on.

The protection of data is a very important responsibility, and includes not just the storage but also the manipulation of data. It is part of our duty as software professionals to make sure that our activities do not undermine the important principle that personal data should be protected.

The 1984 Act is discussed in *Professional Issues in Software Engineering* (1995) by F. Bott, A. Coleman, J. Eaton and D. Rowland.

In the UK, the 1984 Data Protection Act was introduced to try to address this issue. Under the Act, any person (and in legal terms this includes individuals and organisations) who holds data about someone else in electronic form must register with the Data Protection Register. The Act concerns personal data; and, for such data to be governed by the Act, it must be possible to identify the individual from the data stored.

In 1998 a new Data Protection Act passed into law, in response to the 1995 EC Data Protection Directive, bringing manual records into its scope as well. The 1984 and 1998 Acts enforce the following eight data protection principles.

1 Personal data shall be processed fairly and lawfully, and only when certain scheduled conditions do not apply.

2 Personal data shall be obtained only for one or more specified and lawful purposes, and shall not be further processed in any manner incompatible with that purpose or those purposes.

3 Personal data shall be adequate, relevant and not excessive in relation to the purpose or purposes for which they are processed.

4 Personal data shall be accurate and, where necessary, kept up to date.

5 Personal data processed for any purpose or purposes shall not be kept longer than is necessary for that purpose or those purposes.

6 Personal data shall be processed in accordance with the rights of the data subjects under the Act. (Thus an individual is entitled to access the personal data, to prevent its processing if likely to cause damage or distress, to prevent its use for direct marketing, and to have its processing explained.)

7 Appropriate technical and organisational measures shall be taken against unauthorised or unlawful processing of personal data and against accidental loss or destruction of, or damage to, personal data.

8 Personal data shall not be transferred to a country or territory outside the European Economic Area unless that country or territory ensures an adequate level of protection for the rights and freedoms of data subjects in relation to the processing of personal data.

The 1998 Act is discussed in the British Computer Society's 1998 booklet *Data Protection – Everybody's Business: A Practical Guide for Professionals and Business Managers*, and online at http://www.bcs.org/server.php?show=ConWebDoc.2751 (Accessed 26 February 2008).

Computer misuse

Another legal issue arising from the storage of information on computers is **hacking**; that is, unauthorised access to computer-stored information. You may think that hacking is obviously illegal, but the UK's House of Lords ruled that a hacker who worked his way into the mail-box of the Duke of Edinburgh but did not alter any data was not acting illegally. This issue gave rise, eventually, to the 1990 Computer Misuse Act in the UK, which introduced three offences:

- unauthorised access to data or programs with no intention to act further;
- unauthorised access to data or programs with the intention to commit a crime (it may be that the intended crime does not involve a computer, or indeed that the intention is never carried through);
- unauthorised modification or destruction of data or programs.

Some people use the term 'hacking' to mean individual heroic or elegant programming. However, the term is more often used to mean unauthorised access to computing systems, and that is how we use it here.

In all cases, the access must be wilfully committed. In other words, accidental or inadvertent accessing of information does not break the law. The first offence is intended to deter hackers who think it might be interesting or fun to 'eavesdrop' on some information; it is hoped they will be put off if they know that even unauthorised access is in itself illegal. The second offence is a more serious crime and carries a heavier penalty. Gaining access to personal data with the intention of blackmailing an individual would fall into this second category. The third is the most serious offence. Changes to personal data could be very harmful for the individual concerned. Also, if someone hacked into a safety-critical piece of software and deliberately modified some of the safety-related areas, the results could be catastrophic.

Other acts which are classed as misuse of computers, and hence are covered by the Computer Misuse Act, include computer fraud (that is, the use of a computer dishonestly to cause the gain or loss of anything of any value), the denying of authorised access to a computer, and the unauthorised removal of information from a computer.

Health and safety

In May 1990, the European Community Council issued a Directive concerning the minimum health and safety requirements for the use of display screen equipment. The issues covered include the display screen itself, the keyboard, the software interface, the task to be completed and the physical environment in which the task is to be completed. Directives from the European Community are part of its legislative powers and are binding for member states. The results specified must be achieved, but exactly how they are to be achieved is left to the national authorities in each state. In the UK, this authority is the Health and Safety Executive (HSE), which has picked up this Directive and translated it into law. This is of relevance to software professionals in terms of both their own conditions of work and those they impose on their users.

The document was published in 1992 by HMSO.

The following extract comes from Annex A of *Display Screen Equipment at Work*: *Guidance on Regulations* produced by the HSE. It relates to Regulation 3: requirements for workstations.

Principles of software ergonomics

In most display screen work the software controls both the presentation of the information on the screen and the ways in which the worker can manipulate the information. Thus software design can be an important element of task design. Software that is badly designed or inappropriate for the task will impede the efficient completion of the work and in some cases may cause sufficient stress to affect the health of a user. Involving a sample of users in the purchase or design of software can help to avoid problems.

In many (though not all) applications the main points are:

Suitability for the task

- Software should enable workers to complete the task efficiently, without presenting unnecessary problems or obstacles.

Ease of use and adaptability

- Workers should be able to feel that they can master the system and use it effectively following appropriate training.
- The dialogue between the system and the worker should be appropriate for the worker's ability.
- Where appropriate, software should enable workers to adapt the user interface to suit their ability level and preferences.
- The software should protect workers from the consequences of errors, for example by providing appropriate warnings and information and by enabling 'lost' data to be recovered wherever practicable.

Feedback on system performance

- The system should provide appropriate feedback, which may include error messages; suitable assistance ('help') to workers on request; and messages about changes in the system such as malfunctions or overloading.
- Feedback messages should be presented at the right time and in an appropriate style and format. They should not contain unnecessary information.

Format and pace

- Speed of response to commands and instructions should be appropriate to the task and to workers' abilities.
- Characters, cursor movements and position changes should where possible be shown on the screen as soon as they are input.

Performance monitoring facilities

- Quantitative or qualitative checking facilities built into the software can lead to stress if they have adverse results such as an over-emphasis on output speed.
- It is possible to design monitoring systems that avoid these drawbacks and provide information that is helpful to workers as well as managers. However, in all cases workers should be kept informed about the introduction and operation of such systems.

SAQ 15

Look at your current or a previous contract of employment or appointment, and identify what, if anything, is said about the ownership of anything you produce, about the people you contact, or about the expertise you gain during the period of your employment.

ANSWER.....

The contract may say nothing, but it is common for employers to stipulate that everything created by you while in their employment belongs to the employer. Other employers might stipulate that a creation belongs to the employer only if it is related to the employer's business. It is also quite common to stipulate that you should not approach any clients of your employer for custom within two years of leaving the company.

SAQ 16

Suggest an area of software development, other than those mentioned in the text, that causes problems with regard to software ownership.

ANSWER.....

One of the difficult areas at present is class libraries for use in object-oriented software development. Class libraries are being sold, but there are outstanding issues about who is responsible for faults found in the class libraries, whether libraries can be modified and sold on to others, whether royalties should be paid to the original developers of class libraries, and so on.

SAQ 17

If the description of a new, innovative software product is placed on the internet, could this become the subject of a patent? Is it covered by copyright?

ANSWER.....

In terms of patenting, the key point here is whether or not placing items on the internet is regarded as publishing, since the subject of a patent must not have been published. Placing items on the internet is viewed as publishing. Unless the copyright has been signed away in some other agreement, the copyright of the expression of the idea will rest with the originator (but note that this does not cover the idea itself).

SAQ 18

What is the difference between freeware, public-domain software and shareware?

ANSWER.....

Freeware is software that is given away free; copyright remains with the originator.

Public-domain software is also given away free; the originator signs away their copyright.

Shareware is software that is supplied free of charge for a limited time; copyright remains with the originator.

SAQ 19

How many organisations do you think have personal data about you held on a computer?

ANSWER.....

In thinking about this, you may have included, for example, the tax office, your bank, your employer, any clubs you belong to, The Open University, and the Vehicle Licensing Authority. If you have a subscription for a magazine or newspaper, this information may be held on computer. If you have a loyalty card of any kind, personal data about you is likely to be stored electronically by the company concerned. The list goes on and on.

SAQ 20

Why do you think that the third principle of the 1998 Data Protection Act states that stored personal data must be adequate, relevant and not excessive? What might happen if the stored data was not adequate, not relevant and/or excessive?

ANSWER.....

If the data is to be used as the basis of decision making of any kind and it is not adequate, then an inappropriate decision may be made. If the information is not relevant or is excessive, this may also lead to an inappropriate decision, and it could also lead to a breach of privacy.

SAQ 21

Is the deliberate introduction of a virus into a piece of software illegal in terms of the three offences created by the Computer Misuse Act?

ANSWER.....

Yes, it is unauthorised modification (and ultimately, maybe, the destruction) of data and/or programs. (Note that viruses are often passed inadvertently, and this is not regarded as a crime.)

Exercise 12

Consider the following scenario.

This scenario is based on Weiss (ed.) (1982, pp. 186–7).

A software engineer was given project responsibility to develop a customer billing and credit system for his employer, a large retail business. He initially thought the budget and resources he was given were adequate. However, the budgeted amount was expended before completion of the system. In response to his warnings of impending problems, he was directed to finish the development as soon as possible and at lowest cost. He was forced by management to forgo many of the program functions, including audit controls, safeguards, flexibility, error detection and correction capabilities, automatic exception handling, and exception reporting. A 'bare bones' system was installed. He was told that he could add all the omitted capabilities in subsequent versions, after production of the initial system.

A difficult, expensive and extensive conversion to the new system occurred. After the new system was in production, great problems arose. Many customers received incorrect and incomprehensible billings and credit statements, and became outraged. The retail company was unable to correct errors or explain confusing system output. Fraud increased. Business and profits declined, and customers suffered much anguish and personal expense. The software engineer was blamed for the losses.

Is there an ethical issue involved in this scenario?

Solution

When this scenario was presented to a panel of computer scientists as part of an experiment on ethical conflicts, eight out of 31 participants believed that no ethical issue was involved. (One commented that not enough information was given in the scenario about responsibility to decide whether it was an ethical issue.) We would side with the 23 who claimed that the scenario does raise an ethical issue.

The informal definition of an ethical decision – one that can make you feel uncomfortable – is of some help in justifying our claim, since the engineer, through his repeated warnings, seemed to express his unease. The more formal definition – whether a living thing may be adversely affected emotionally, practically or morally – is perhaps even clearer. The language used to describe the scenario not only indicates the level of emotional anxiety experienced by the engineer, but also that experienced by the managers of the company and its customers. And as well as being emotionally affected, all of these parties were also adversely affected in practical terms: the engineer by being blamed, his company by losing business, and the customers by suffering financial losses.

Exercise 13

Think again about the situation in Exercise 12. An ethical decision arose at the point when the software engineer was being directed to finish the development as soon as possible and at lowest cost. Use the eight-step procedure to identify the possible courses of action, and to decide on the most suitable option.

Solution

1 *Is the issue a symptom of a wider concern?* The issue concerns inadequate financial estimation and/or inefficient management of the resources available. Both of these issues could and should be addressed by the company, but the engineer does not appear to have been in a position to resolve them.

2 *Is the situation governed by law or organisational rules?* No, or not in any way that appears obvious from the description.

3 *Identify any party affected*. The parties affected are the customers, shareholders and employees of the large retail business, including the engineer.

4 *Identify the main options, their justifications, and their expected outcomes*.

   (a) Do as asked by the management and implement a limited version of the system without the functions specified. The potential outcomes (in fact, probably the worst-case outcomes) are reflected in the description of the outcomes presented in Exercise 12.

   (b) Resign. This option would safeguard the engineer's conscience, but the management are likely to find another engineer to carry through their wishes. This course of action does little to address the issue at hand: it is avoidance.

   (c) Work out a compromise situation, providing a safe and reliable system with lower functionality. This option may require more resources, but they could be balanced against the potential losses of a system without the quality and audit

controls originally specified. Present this option to the management as an alternative strategy.

5 *Eliminate options that are 'illegal'*. None of the options appears to fall within this category.

6 *Ask negative questions about the expected outcomes of each option.*

(a) Will any custom be lost? What is the likelihood of fraud? Will the system comply with company policy on quality? Will our ISO 9000 accreditation be compromised if we deliver this system?

However, carrying out an action on the basis that if you don't somebody else will, is often held as poor justification.

(b) Will the engineer get another job? Is not the engineer resigning, in effect, the same as option (a), since another engineer will be brought in to follow the company line? Consequently, what does this option achieve, except short-term gains for the engineer through the avoidance of a difficult decision?

(c) Will an acceptable balance be possible? Will removing functionality be detrimental to the client? Will the new resource estimate be reliable?

7 *Ask neutral or positive questions about the expected outcomes of each option.*

(a) How soon will the additional functionality be available? How much money can be saved by introducing the new system earlier?

(b) Can the engineer be replaced with someone as competent, or more competent, to manage the project?

(c) What would the management lose by delivering a safe, reliable system with lower functionality? What would they gain?

8 *Select a course of action*. The most attractive option seems to be option (c): planning a compromise situation, providing a safe, reliable system with lower functionality. (This option may not have been possible in reality.)

Exercise 14

Consider the following situation.

Company A has offices in different countries around the world. For a particular project, the team is distributed across countries. One part of the team sends program code for part of the software, by email, to another part of the team. This program code is intercepted by Company B, one of Company A's competitors, and released onto the market.

Has Company B infringed Company A's copyright? Suggest the main issues to be considered, and any evidence which might be appropriate. (You might find it helpful to refer to the cases cited from David Bainbridge's book.)

Solution

When considering copyright issues, the main question is whether the idea itself has been copied or whether the expression of the idea has been copied (although note court case 4 cited from Bainbridge's book, in which it was determined that not every expression of an idea is automatically protected by copyright). The way in which the intercepted code was incorporated into the system is pertinent: if Company B incorporated the code directly into their own product, then they have appropriated the expression. However, if they took the structure and recoded the software in a different language, they have appropriated the structure. Note court case 3 from Bainbridge's book, in which it was ruled that infringement depends on the skill and judgement required to select and integrate elements of a structure. Also, court case 4 raised four issues about the expression of an idea that must be considered.

The evidence to be produced would include program listings. Evidence to support the claim that Company B did indeed intercept the email might consist of system logs or other tracing/audit controls.

Exercise 15

Consider the following situation.

A software engineer works for a large software house which specialises in software systems to support financial organisations. In her spare time she enjoys designing and prototyping computer games. Out of this part-time interest, she develops a new kind of computer game, which she believes would make a good commercial product. Since her current employer does not operate in the appropriate market, she strikes a deal with another company which specialises in developing and selling computer games. On hearing about the deal, her employers are furious, but she believes that she has acted within her rights.

Deciding who is right or wrong in this case could form the basis of a protracted lawsuit – something we would not be able to resolve here. However, in trying to sort out the issue, what documents or people might you consult, and what questions might you ask?

Solution

The documents to consult would include the engineer's employment contract. According to the general law of copyright, the engineer would own the copyright on her new computer game, and would therefore be at liberty to sell the game to the highest bidder for her own gain. However, copyright can be assigned, and it is important to discover whether the contract she has with her employer specifically states that all copyrights that would normally rest with her are assigned to her employer. This kind of clause is not uncommon.

A second document might be the agreement signed between the engineer and the second company. This document would show the commitment being made by the engineer to the new company; it would be important to establish whether there were any conflicts between the two contracts.

The main person to consult would be the engineer herself, to ascertain whether any records were kept of the development process that confirm that the development took place in her 'spare time' as opposed to 'company time'. Also, if machinery, software or other equipment used to develop the game belongs to her employers, then they may have a claim on the rights to the product.

Exercise 16

Consider one piece of software that you know well, such as your favourite word processor, project management tool, development environment, or computer game. Comment on how and to what extent this piece of software conforms to the points made under the heading *Feedback on system performance* in the extract from HSE's *Display Screen Equipment at Work* document quoted earlier.

Solution

The solution to this exercise obviously depends on the piece of software chosen, but the appropriateness and accuracy of error messages and the usefulness and suitability of help facilities are often criticised.

5.3 Summary of section

In this section you have learnt that ethical decision making involves choosing between 'right' and 'wrong'. An individual is guided in making ethical decisions by a set of moral principles derived from individual, social and professional influences.

You learnt that the important point about ethical decision making, from a professional's point of view, is that the options and their consequences should be carefully considered and thought through before a decision is taken. An eight-step procedure was proposed that can help to structure this process.

You have also learnt that software professionals should be aware of their legal obligations and rights. In particular, you have studied the legal issues associated with some of the most important areas of software development, such as contracts, software ownership, data protection, computer misuse, and health and safety.

6 Professional bodies

In Section 5 you saw how those involved with developing and using software enjoy rights and incur responsibilities, both in law and by virtue of being professionals. The aim of this section is to study the role of professional bodies and to examine how they support software professionals in exercising their duties.

6.1 The role of professional bodies

There is a wide variety of organisations that a software professional can choose to join. For example, the IET (Institution of Engineering and Technology), the IEEE (Institute of Electronic and Electrical Engineers) Computer Society, the BCS (British Computer Society) and the ACM (Association for Computing Machinery). A number of joint ventures exist to draw these different societies together. For example, the Council of European Professional Informatics Societies (CEPIS) was set up to provide a coordinated voice to express to European institutions the views of European 'informatics professionals' on major issues. Its areas of focus include professional development, public awareness and standardisation. The goals of these different bodies are broadly similar.

Consider the 'about' page on the website for the IEEE Computer Society, which states:

> The Society is dedicated to advancing the theory, practice, and application of computer and information processing technology.
>
> http://www.computer.org/portal/site/ieeecs/index.jsp (Accessed 26 February 2008).

The page then goes on to list many different ways in which the society hopes to fulfil this ambition, including providing information on conferences and books, and links to colleagues and discussion groups around the world. This highlights one of the most important roles of these bodies: to support their members in their professional practice.

As another example, consider the Royal Charter for the BCS, which states that the objectives of the society are:

> The objects for which the Society is hereby constituted shall be to promote the study and practice of Computing and to advance knowledge and education therein for the benefit of the public.
>
> http://www.bcs.org/upload/pdf/royalcharter.pdf (Accessed 26 February 2008).

The Charter goes on to list 25 powers that the society will have. These include powers related to the running and organisation of the society, but also the power to establish and maintain educational standards, to establish and maintain an ethical foundation for the use of computers, to confer with other organisations in pursuit of its own objectives, to prepare and publish books and journals, and to found, aid, maintain and endow bursaries to support individuals engaged in computing.

As it is not obligatory for a software professional to belong to a professional body, such bodies are not given the power to prevent particular individuals from practising. However, they can and do maintain publicly available registers of people qualified in certain fields who fulfil minimum requirements. They are also able to refuse or withdraw membership from those who do not fulfil appropriate requirements. In practice, the

standards of behaviour and training set for membership of these bodies act as an implicit yardstick for those inside and outside the profession.

Professional bodies may represent the profession, supplying impartial advice and expertise on matters relating to software, including legislation, public inquiries, and so on. They also sometimes act as the voice of the profession in debates on relevant issues, such as whether or not software professionals should be regarded as 'engineers'.

Probably the most obvious support mechanism offered to anyone who joins a professional body is the ready supply of information. The only way in which professionals can keep their knowledge up to date is through information: information about new books, forthcoming events, new products, standards, developing issues and so on. Professional bodies form an obvious channel for this information. Seminars, conferences and meetings provide an opportunity for members to meet together and discuss 'hot' topics and new developments in their field of interest. Special-interest groups (SIGs) allow people interested in one aspect of the field to get together more often and provide a focus for specific activities. For example, the ACM has special-interest groups on graphics (SIGGRAPH), information retrieval (SIGIR), programming languages and notations (SIGPLAN), human–computer interaction (SIGCHI), computer science education (SIGCSE), and many other topics.

Some more formal approaches to encouraging this process of disseminating information also exist. For example, the BCS runs a professional development scheme that encourages members to keep up to date and upgrade their knowledge and skills. To support it, they have also issued an industry structure model (ISM), which is a set of performance standards designed to cover all functional areas of work carried out by practitioners and professionals within the province of the BCS. The ISM is intended to be a set of performance yardsticks that can be used to identify training needs and design training plans for individuals wishing to gain membership of professional bodies.

Information is not the only source of support. Professional bodies also provide advice and encouragement on more sensitive matters. For example, on joining a professional body, an individual agrees to abide by any codes of practice or codes of conduct upheld by the body. These and other professional codes can provide invaluable support and advice, and are discussed in the next subsection.

SAQ 22

List the three key roles of professional bodies in software development.

ANSWER.....

1 To provide support to help members fulfil their professional duties.

2 To set standards of behaviour and training for their members in order to provide a realistic yardstick for those inside and outside the profession.

3 To represent the profession in debates on relevant issues.

SAQ 23

How does the provision of information help to improve professional competence?

ANSWER.....

Articles, seminars, workshops, meetings, books and so on can be used to report on the practical experiences of members.

6.2 Codes of conduct and codes of practice

As an example of the kind of rules governing members of a professional organisation, we consider here the *Code of Conduct* and the *Code of Good Practice* for BCS members. The two documents are complementary: the *Code of Conduct* deals with the way in which members of the Society should behave; the *Code of Good Practice* deals with the way in which members should exercise their professional competence. These codes are accessible on the course website via the web page for this unit (the relevant website is listed under the heading BCS Codes). Read through the codes.

Cartoon by Gordon Cunningham © The Open University.

Figure 5 Professionalism means accepting responsibility for all aspects of software development

In reading through the codes, it is possible to identify areas where they overlap. For example, the personal requirements on a member to 'Keep up to date with technological advances, through training, technical publications and specialist groups within professional bodies', as stated in the *Code of Good Practice*, complements the *Code of Conduct* rules regarding the upgrading of a member's knowledge and skill.

Exercise 17

Seek out an example of a publication produced by one of the professional bodies (ACM, IEEE, BCS or IET) mentioned in the text, through your library, your tutor, your workplace, or your own personal collection. Look through it and identify the different professional issues it addresses.

Solution .....

The result of this exercise will depend on the publications to which you have access. The issues covered will range from general updating on products and technologies to

specialist papers relating to new ideas emerging from research, and will include lists of upcoming conferences and meetings, job adverts and opinion columns.

Exercise 18

Read once more through the BCS *Code of Good Conduct* (accessible via the web page for this unit) and then look back at the scenario discussed in Exercises 12 and 13. Identify which rules of professional conduct address issues relevant to this scenario, and how adherence to these rules would affect the software engineer's behaviour.

Solution

- Rule 2 is relevant, in that the customers of his employers have the right to have correct bills.
- Rule 3, regarding legislation, may be relevant if there is a statutory requirement for audit controls to be included in the system.
- Adherence to Rule 6 means that the engineer should indicate the consequences of not including all of the functions.
- Rule 9 may have been breached, in that users and customers of the system presumably believed that the system's capabilities were other than they turned out to be.
- Rule 10 was breached. Any inadequate software system that is installed casts doubts on the profession as a whole.
- Rule 17 is about professional responsibility. The engineer was blamed for the problems, but if the consequences of installing a skeletal system had been communicated to the management his professional duty would have been fulfilled.

Exercise 19

Consider a current or recent project with which you have been engaged that involves software: it might be a software development project, a project in which software is used for support, or a software maintenance project. Study the BCS *Code of Conduct* and relevant sections of the BCS *Code of Good Practice* (accessible via the web page for this unit) and consider how each rule or statement would affect your behaviour or decisions.

Solution

The details of your deliberations will depend on the project you have chosen. As with ethical decision making, and many other professional issues, the important thing is that you have studied the codes and have thought carefully about their applicability in a specific situation.

Exercise 20

Look up, on the internet, the BCS, the IET and any other professional body you choose, and see what they say about professionalism.

The website addresses for the BCS and IET can be found on the web page for this unit under the headings BCS and IET.

Solution

You will find much common ground. If you look up a US body, such as the ACM or the IEEE, you will find more emphasis on information sharing and less on certification of individuals than for a UK body.

6.3 Summary of section

In this section you have learnt about the role of professional bodies in representing the profession, in setting standards of behaviour, and in providing support to help members fulfil their professional duties.

You have studied in some detail the BCS *Code of Conduct* and *Code of Practice,* and have seen how they govern the way professionals should behave and exercise their professional competence. You have also seen that these codes can provide guidance in making ethical decisions.

7 Summary

This unit has been about the human factors that affect the software development process and what it means to be a professional software engineer. You saw that the human factors could be categorised as either process-related or product-related, and that, for the former, you should be able to distinguish between person-related, task-related and environmental-related factors. You saw that professional software engineers must carefully consider the ethical and legal issues involved in software development (an eight-step procedure was given to help structure the consideration of ethical issues), and that professional bodies represent the profession by setting standards and providing support to their members in fulfilling their professional duties.

Regarding human factors affecting software development, you learnt that:

- software development is carried out by individuals grouped together as teams;
- competence and motivation are two key attributes for software engineers;
- the social dimensions of work are influential in determining the success or failure of a project;
- environmental factors can affect performance;
- some tasks can be automated, whereas others are more suited to humans.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- explain the purpose of an analysis of human factors;
- distinguish between product-related and process-related human factors;
- distinguish between person-related, task-related and environment-related human factors;
- describe the three cornerstones of professionalism;
- describe how certain aspects of professionalism apply to software engineers;
- identify important domains of competence for any software engineering role and suggest how competence in these domains can be enhanced;
- analyse the aspects of teams that affect their performance;
- identify task characteristics that are likely to limit an engineer's performance;
- identify environmental characteristics that can affect human performance;
- explain why some tasks are suitable for automation and some are not;
- recognise a decision involving ethical issues;
- tackle a situation involving ethical decision making by following an eight-step procedure;
- describe some of the main contractual issues surrounding software development, and explain their importance;
- discuss the notion of the ownership of computer software, recognising the problems that it poses, and develop potential solutions to those problems;
- distinguish between patent and copyright, and identify the situations in which each is more appropriate;
- explain the need for, and the implications of, data protection legislation;

- recognise an action that would be classified as computer misuse;
- apply informally to a piece of software the Health and Safety Executive's regulations relating to display screen equipment;
- describe the key roles of professional bodies;
- apply the BCS *Code of Conduct* and *Code of Good Practice* to example situations.

References

Bainbridge, D. (1994) *Software Copyright Law*, 2nd Edn, Butterworths.

Bott, F., Coleman, A., Eaton, J. and Rowland, D. (1995) *Professional Issues in Software Engineering*, 2nd Edn, UCL Press.

Conger, S.A. (1994) *The New Software Engineering*, Wadsworth Publishing Company Inc.

Conway, M.E. (1968) 'How do committees invent?', *Datamation*, vol. 14, no. 4, pp. 28–31.

DeMarco, T. and Lister, T.R. (1987) Peopleware: Productive Projects and Teams, Dorset House Publishing Co.

Hackman, J.R. and Oldham, G.R. (1976) 'Motivation through the design of work: test of a theory', *Organizational Behavior and Human Performance*, vol. 16, no. 2.

Kallman, E.A. and Grillo, J.P. (1996) *Ethical Decision Making and Information Technology: An Introduction with Cases* (2nd Edn), McGraw-Hill.

Langford, D. (1995) *Practical Computer Ethics*, McGraw-Hill.

Leveson, N. and Turner, C. (1993) 'An investigation of the Therac-25 accidents', *IEEE Computer*, vol. 26, no. 7, pp. 18–41.

McCue, G. (1978) 'Architecture design for program development', IBM Systems Journal, vol. 17, no. 1.

The British Computer Society (1998) *Data Protection – Everybody's Business: A Practical Guide for Professionals and Business Managers*.

Weiss, E.A. (ed.) (1982) 'Self-assessment procedure IX: a self-assessment procedure dealing with ethics in computing', *Communications of the ACM*, vol. 25, no. 3, pp. 186–7.

Acknowledgements

Text

Health and Safety Executive (1992) *Display Screen Equipment at Work. Guidance on Regulations.*

Crown copyright material is reproduced under Class Licence Number C01W0000065 with the permission of the Controller of HMSO and the Queen's Printer for Scotland.

Index